

ARCHER2 上 PETSc Krylov 求解器的 混合 MPI+OpenMP 性能

Leyan Li

高性能计算理学硕士
爱丁堡大学 EPCC

2026 年 8 月

本论文为部分满足理学硕士学位要求而提交。

摘要

PETSc 可以在 MPI rank 内使用 OpenMP 线程，但最佳配比取决于求解器、数据分解和并发规模。本文在 ARCHER2 上比较四种占满节点的布局：纯 MPI，以及每个 rank 使用 2、4 或 8 个线程的混合布局。配对的二维和三维 Poisson 基准覆盖 5.0–164.6 million 未知量和 1–32 个节点。每个正式配置运行三次；一次预热求解建立 GAMG 层次结构，随后 19 次计时求解复用该结构。筛选后保留的 11 个规模–节点组合构成两条近似保持每核工作量不变的弱扩展链。

二维结果随并发规模呈较稳定的次序：六条“布局–扩展链”序列中有四条单调改善。 64×2 在全部 11 个组合上的单次迭代时间都短于纯 MPI；在 32 个节点、164.56 million 未知量时，其原始吞吐率高 13.1%。三维的最大优势更大，达到 33.2%，但六条序列均不单调，首选布局随问题规模和节点数变化。在保持每核工作量的实验中，每 rank 八线程在两个维度都没有取得最高吞吐率。

配对的扩展实验把问题规模固定在约 20 million 未知量，并从 1 个节点扩展到 32 个节点。四种二维布局在 4096 核时仍继续改善。三维中，纯 MPI 在 8 个节点后转差，两种中间布局在 16 个节点后转差，而 16×8 到 32 个节点仍在改善，并比同一节点数下的纯 MPI 快 $3.81\times$ 。PETSc 事件数据表明，混合布局的优势与较低的 halo exchange 和全局归约暴露度相关。11 份 Linaro MAP profile 显示，随着线程替代 rank，OpenMP runtime 中的等待比例反向上升；线程化 BLAS 最多只占求解期间 core time 的 2.1%。这些测量定位了软件成本，但没有确定底层硬件机制。

致谢

Contents

摘要	i
致谢	ii
缩略语	ix
1 引言	1
1.1 研究动机	1
1.2 研究目标与研究问题	1
1.3 主要贡献	1
1.4 论文结构	2
2 背景与相关工作	3
2.1 并行编程模型	3
2.2 纯 MPI 的限制	3
2.3 混合 MPI+OpenMP	4
2.4 ARCHER2 节点架构与 NUMA	4
2.5 PETSc 与 Krylov 求解器	6
2.6 性能分析工具	6
2.7 小结与研究空缺	7
3 实验平台与方法	8
3.1 硬件和软件环境	8
3.2 基准问题与驱动程序	8
3.3 求解器配置	9
3.4 配置空间与筛选规则	10
3.5 限定线程数	10
3.5.1 探索性单节点 sweep	10
3.5.2 生产配置中的相同线程上限	12
3.6 作业配置与线程放置	12
3.7 性能指标	12
3.8 已废弃的实验 campaign	13
3.9 可复现性	14
4 2D 与 3D 问题	15
4.1 实验设置	15
4.1.1 弱扩展链	15
4.1.2 强缩放设计	16
4.2 每核工作量恒定时的弱扩展	16
4.2.1 2D	16
4.2.2 3D	18
4.2.3 布局排名与效率	19

4.3	固定问题规模下的强扩展	20
4.3.1	2D	20
4.3.2	3D	21
4.3.3	端点比较	22
4.4	综合解释	23
5	性能剖析与瓶颈分析	24
5.1	为何使用重复求解驱动程序	24
5.2	分析方法	24
5.3	MAP profiling 结果	24
5.4	瓶颈识别	26
5.4.1	测量什么	26
5.4.2	Halo 交换主导暴露量, 混合并行降低暴露量	27
5.4.3	降低暴露量是必要条件, 但并不充分	29
5.4.4	证据能够与不能支持的结论	32
6	讨论	33
6.1	回答研究问题	33
6.1.1	RQ1: 线程后端是否缩短求解时间, 幅度多大?	33
6.1.2	RQ2: 哪种配比更好, 结论如何随扩展场景变化?	33
6.1.3	RQ3: 求解过程的哪些部分与性能差异相对应?	34
6.1.4	RQ4: 这些趋势可推广到何种范围, 还需哪些证据才能确定机制?	34
6.2	有效性威胁	34
7	结论与未来工作	36
7.1	结论	36
7.2	ARCHER2 上 PETSc 的实践建议	36
7.3	未来工作	37
	Bibliography	38
A	可复现性: 仓库与脚本	39
A.1	目录结构	39
A.2	证据索引	39
A.3	重新生成结果	41
B	补充图表	43
B.1	弱扩展吞吐量	43
B.2	固定规模运行时间网格	44
C	完整 log_view 摘录	46

List of Figures

2.1	ARCHER2 计算节点的层次结构。每节点包含两个 64 核 AMD EPYC 7742 处理器、八个 16 核 NUMA region, 以及各自共享 16 MB L3 cache 的四核 CCX。来源: ARCHER2 硬件文档 [8].	5
3.1	PETSc 默认的行分块如何决定子域的形状。词典序中连续的行使每个 rank 的子域成为若干完整 $m \times n$ 平面的堆叠, 因此它的 halo 就是与 k 方向两个邻居共享的那两个平面; 一旦 L 降到 mn 以下, halo 就不会低于 $2L$ 。笛卡尔分解则在每个方向上都切一刀, 得到六个更小的面。本文所有运行都使用 (a), (b) 是 Section 7.3 提出的替代方案。深色块是某一个 rank 的子域, 深色表面是它需要交换的部分。	9
3.2	1000 $\times$ 1000 网格上的单节点默认求解器 sweep。在单个 MPI rank 中增加 OpenMP 线程时, 每次迭代成本几乎不变; 保持每 rank 单线程并增加 MPI rank 时, 成本下降两个数量级。两条线均从同一个单 rank、单线程运行开始。	11
4.1	2D time per KSP iteration relative to the 128×1 baseline at the same size-node pair; values below 1 mean the hybrid layout is faster. Whiskers propagate the observed range of three formal runs.	17
4.2	Median 2D time per KSP iteration along the two unknowns/core chains. A horizontal line would be ideal weak scaling.	17
4.3	3D time per KSP iteration relative to the 128×1 baseline at the same size-node pair. Unlike the 2D case in Figure 4.1, no series is monotone.	18
4.4	Median 3D time per KSP iteration along the two unknowns/core chains.	19
4.5	2D time per solve at a fixed 20.25M unknowns, one to 32 nodes. Both axes are logarithmic, so ideal strong scaling is a straight line of slope -1 . Whiskers show the observed range of three formal runs.	21
4.6	3D time per solve at a fixed 20.35M unknowns. Flat MPI bottoms at eight nodes, 64×2 and 32×4 at 16; only 16×8 still improves at 32.	21
4.7	3D parallel efficiency at a fixed 20.35M unknowns, each layout against its own one-node time. Because the baselines differ, this figure ranks scaling behaviour and not speed; flat MPI falls to 10.9% at 32 nodes while 16×8 retains 44.5%.	22
5.1	Where the cores are during the solve, from the MAP call trees. Within each panel the OpenMP waiting share grows and the MPI share shrinks as threads replace ranks, and both effects strengthen with node count. Paired bars are the two PETSc builds at the same configuration.	25
5.2	2D share of <code>RepeatedSolves</code> KSPSolve time spent inside halo exchange and global reduction events, panelled by unknowns/core chain as in Chapter 4. Only the 40k chain reaches 32 nodes. Each point is the median of three repetitions. The ordering matches the time-per-iteration ordering in Figure 4.1 at every node count.	27
5.3	3D share of <code>RepeatedSolves</code> KSPSolve time spent inside halo exchange and global reduction events, panelled and styled as in Figure 5.2. Note the vertical scale: flat MPI reaches 84.6% at the largest point.	28

5.4	3D exposure removed relative to flat MPI against the resulting time per iteration. Colour is the layout and point labels are node counts. The solid line is the pooled fit over all 11 points, the one Table 5.4 tabulates; the two broken lines split those points into their weak-scaling chains, dashed with filled markers for $\sim 40k$ unknowns/core and dash-dotted with hollow markers for $\sim 80k$. In 3D the three stay together, every layout's per-chain break-evens falling within 2.4 points of its pooled value. Each line is clipped to the range it was fitted to, so none is extrapolated; the fits are descriptive, not predictive.	30
5.5	2D exposure removed relative to flat MPI against the resulting time per iteration, styled as in Figure 5.4. Here the solid pooled fits are not summaries of their parts. For 16×8 the solid line lies flat while its two chains, correlating at -0.91 and -0.96 separately, descend steeply across it; 64×2 and 32×4 depart from their pooled fits as well. Note also that 16×8 lies above parity at every one of its 11 points, which is read off the markers and not off any fit.	31
B.1	Median 2D repeated-solve throughput along the two unknowns/core chains. Whiskers show the observed range of three formal runs. All layouts use 128 cores per node, so a fixed node count means a fixed core count.	43
B.2	Median 3D repeated-solve throughput along the two unknowns/core chains. Whiskers show the observed range of three formal runs.	44

List of Tables

2.1	三种执行模型在一个 ARCHER2 节点内的含义。混合模型同时承担 MPI 和 OpenMP 成本。	3
2.2	Close binding 下的四种满节点布局。它们均使用 128 个物理核心，但 rank 数和相对共享 L3 边界的位置不同。该表只描述放置，不断言性能机制。	5
2.3	一次 GAMG 预条件 CG step 中的 PETSc event 和实测调用数。一个 step 对应一次预条件器应用；CG 会先预条件初始残差，因此每次求解的 step 数比报告的迭代数多一。	6
3.1	1000 × 1000 网格上单节点默认求解器 sweep 的最终迭代数，覆盖所有 ranks × threads ≤ 128 组合。每行恒定，说明迭代数只由 rank 数决定。128-rank 配置达到 10,000 次上限且未收敛。	10
3.2	Cost per KSP iteration for the eight fully populated single-node layouts (ranks × threads = 128) on a 1000 × 1000 grid. The trend is monotone in the rank count, with no threshold at any particular thread count. The 128 × 1 figure is measured over its non-converged 10,000 iterations, which still gives a valid per-iteration cost.	11
3.3	KSP time per iteration for five full-node layouts in the production configuration. Each cell has one run and every row rises with threads per rank. The campaign has no 128 × 1 cell and bounds the search space rather than ranking the admitted layouts.	12
3.4	Relative tolerance actually used by ex2 on a 100 × 100 grid, read from -ksp_view, for the three option styles. The double dash is silently ignored and the mesh-dependent default survives; -options_left names the unconsumed option explicitly.	13
4.1	Grids and node counts. Weak-scaling points pass the unknowns-per-core filter of Section 3.4; the fixed-size campaign runs the 20m problem over all nodes.	15
4.2	The 11 admitted pairs arranged as two weak-scaling chains with near-constant unknowns per core.	16
4.3	Fastest layout by median throughput over three repeats, its margin over flat MPI, and time per iteration. The 2D 41m/8-node row is a tie because ranges overlap.	19
4.4	Weak-scaling efficiency (%) at the largest node count of each chain, relative to the same layout's one-node time. Each layout is measured against itself, so this quantifies degradation, not absolute speed.	20
4.5	Fastest layout for each fixed 20M problem, by three-run median time per solve, and its advantage over flat MPI.	22
4.6	The 32-node endpoint of the fixed-size curves. Speedup and cumulative efficiency are measured against the same layout's one-node time, so they reward a poor baseline; the doubling efficiency is the 16-to-32-node step alone. Time per solve is the only column that compares layouts directly.	23
4.7	Cross-dimensional weak- and strong-scaling summary. Throughput gain and time saving use rate and time conventions respectively. A layout turns upward when its time after its measured minimum increases.	23

5.1	Per-event time and flop shares, RepeatedSolves stage, 20.25M, 16×8	24
5.2	Share of total core-time inside the solve window, by the library owning the executing frame, for the eleven usable MAP profiles at 20.25M unknowns in 2D. OpenMP is the GNU runtime's spin and wait paths, that is threads with no work. MAP instrumentation inflates these runs, so the ordering between layouts is the result. The absolute values are not transferable to the uninstrumented campaigns.	25
5.3	The exposure share of Figure 5.2 and Figure 5.3 split into its two event classes, at four representative points along the $\sim 40k$ unknowns/core chain of Table 4.2, for the 128×1 and 16×8 layouts. Halo is <code>VecScatterBegin</code> and <code>VecScatterEnd</code> ; reduction is <code>VecTDot</code> and <code>VecNorm</code> . Their sum is the quantity plotted, so it is left to the figures. Each row is the repetition whose total is the median of its three, so both components come from the same run; the two are rounded independently and at one decimal need not sum to the plotted total (3D, 32 nodes, 128×1 : 70.9 and 13.6 against a plotted 84.6). Percentage points of <code>RepeatedSolves</code> <code>KSPSolve</code> time.	28
5.4	ExposureRemoved against TimeRatio, both relative to flat MPI at the same size–node point, the first in percentage points. The break-even value is the exposure reduction the fitted line requires to offset threading overhead; a and b are that line's slope and intercept, tabulated as $10^3 a$, so $(1 - b)/a$ reproduces the last column. <i>None</i> marks $a \geq 0$, where the line never descends to parity and no break-even exists. Eleven points per layout; the fits are descriptive only.	29
A.1	Repository layout. Raw logs and retrieved job CSVs are immutable evidence; everything under a <i>derived</i> heading is regenerated by the scripts.	39
A.2	Evidence index. Script paths are relative to <code>analysis/scripts/</code> , derived and raw paths to <code>analysis/</code> . Log counts are the files present, which for the 3D fixed-size campaign exceeds the 72 accepted runs because superseded job attempts are retained. The last two rows are read directly from a named file and have no derived stage.	40
B.1	2D fixed-size campaign, 4500^2 and 20.25M unknowns, median seconds per solve over three runs at every layout and node count. The final columns give the 16-to-32-node speedup and the largest observed three-repeat spread on that row. All layouts fill the node, so 32 nodes is 4096 cores in every row. Iteration counts range from 9 to 11.	44
B.2	3D fixed-size campaign, 273^3 and 20.35M unknowns, median seconds per solve over three runs at every layout and node count. Columns as in Table B.1. A 16-to-32-node figure below 1.00 means the layout is slower at 32 nodes than at 16. Iteration counts are 6 or 7 throughout.	45

缩略语

AMG	代数多重网格
BLAS	基础线性代数子程序，稠密向量和矩阵运算的标准接口
CCD	Core Complex Die; AMD Zen 2 处理器上的两个 CCX
CCX	Core Complex; 四个核心共享一份 16 MB L3 cache
CG	共轭梯度法
GAMG	PETSc 的代数多重网格预条件器 (PCGAMG)
GMRES	广义最小残量法, PETSc 的默认 Krylov 求解器
ICC	不完全 Cholesky 分解
KSP	PETSc 的 Krylov 子空间求解器对象
LAPACK	线性代数程序包, BLAS 之上的稠密分解层
LibSci	HPE Cray Scientific Libraries, 厂商优化的 BLAS 和 LAPACK
MAP	Linaro MAP, Chapter 5 使用的采样 profiler
MPI	消息传递接口
NUMA	非一致内存访问
OpenMP	Open Multi-Processing, 共享内存线程接口
PC	PETSc 的预条件器对象
PETSc	Portable, Extensible Toolkit for Scientific Computation
SMT	同时多线程

Chapter 1

引言

1.1 研究动机

稀疏迭代求解器占据许多科学应用的大部分运行时间，PETSc [1, 2] 是实现这类求解器的常用库。从 3.22 版开始，PETSc 可以在分布式内存 MPI 模型中启用线程化 kernel [3]。因此，在固定核心数下，用户可以选择纯 MPI，也可以让每个进程使用多个 OpenMP 线程。

两种选择都有成本。一个 ARCHER2 计算节点包含 128 个物理核心，分布在两个 socket、八个 NUMA region 和 32 个四核共享 L3 domain 中。纯 MPI 把每个子域表示为独立进程，每个进程都有自己的地址空间、消息匹配和求解器元数据。用线程替代部分 rank 可以减少 MPI endpoint 和子域边界，但会增加线程团队和同步开销，并让同一 rank 内的核心共享 cache 和内存通道。

仅凭核心数无法确定最佳平衡。可节省的时间取决于一次迭代中暴露于通信的比例，而该比例随问题规模、stencil、预条件器和节点数变化。线程成本则取决于每个并行区域中的工作量及其在内存层次结构中的位置。本文直接测量这一权衡，不预设哪一侧占主导。

1.2 研究目标与研究问题

本研究针对配对的二维和三维 Poisson 问题，比较纯 MPI 与三种混合布局。六个问题规模覆盖 5.0–164.6 million 未知量和 1–32 个满载节点；每次比较都固定核心数、求解器和收敛条件。每核未知量筛选使每个规模只保留一到两个节点数，因此这些点可以组成弱扩展链，却不能形成足够长的饱和曲线。为此，配对扩展实验把约 20 million 未知量固定在整个节点范围上。两组实验共同回答以下问题。

RQ1 在节点满载时，PETSc 的线程化 kernel backend 能否相对于纯 MPI 缩短求解时间，幅度有多大？

RQ2 哪些 rank–thread 配比有效？答案如何随问题规模、节点数、维度和扩展模式变化？

RQ3 求解过程中的哪些部分对应这些差异？

RQ4 观察到的趋势可推广到什么范围？确认其机制还需要哪些证据？

Chapter 4 回答 RQ1 和 RQ2，Chapter 5 回答 RQ3，Chapter 6 讨论 RQ4。可行性研究预计每 rank 两到四线程会带来收益，并把 VecMDot 和 PCApply 列为候选限制。之后求解器从 GMRES 改为 CG，VecMDot 也相应变为 VecTDot；Section 3.3 说明了原因。

1.3 主要贡献

1. **配对的满节点 rank–thread 研究**。经验证的二维和三维实验覆盖四种布局、六个问题规模、1–32 个节点和三次正式重复，并包含配对的固定规模扩展。每份日志均根据其内部报

告的 build、进程数、线程数、收敛状态、重复求解阶段和链接库进行检查，而不是依据文件名判断。

2. **并发规模会改变布局排序的证据。**二维混合布局的优势通常沿弱扩展链增长；三维的首选布局则随规模和节点数变化。小节点数上的比较不能直接推广到更大 allocation。
3. **扩展模式会改变建议的证据。**每 rank 八线程没有赢得任何弱扩展点，但固定三维问题扩展到 4096 核时， 16×8 因饱和更晚而成为最快布局。
4. **权衡两侧的事件级证据。**在全部 264 次正式弱扩展运行中，PETSc 事件数据把混合布局与较低的 halo 和归约暴露度联系起来；11 份 MAP profile 则把反向增长的采样成本定位到 OpenMP runtime 等待路径。该联系是描述性的：它识别了软件事件，但不能确定 cache、内存带宽、负载不均衡或并行区域长度是否造成等待。
5. **重复求解测量方法。**Driver 只支付一次 GAMG setup 成本，随后在独立的 PETSc logging stage 中计时 19 次复用求解，避免层次结构构建和 first-touch 效应主导布局比较。
6. **可复现、可审计的 pipeline。**原始日志、分析脚本和图表生成流程一并保留。Pipeline 直接解析 MAP 文件，验证实验完整性，并拒绝无效的 launcher、求解器和 tolerance 比较。脚本断言把表中性质与正文结论绑定；Section 6.2 记录现有证据无法支持的内容。

1.4 论文结构

Chapter 2 介绍执行模型、ARCHER2 topology、CG+GAMG 求解器和 profiling 工具，并说明研究空缺。Chapter 3 定义 benchmark driver、可接受配置、线程放置、指标和验证契约。Chapter 4 分别给出固定每核工作量的弱扩展和固定问题规模的强扩展，不混合两者的证据。Chapter 5 分析 PETSc 事件和 MAP call tree。Chapter 6 回答研究问题并说明有效性威胁。Chapter 7 总结结论、给出实践建议，并列出仍需完成的实验。

Chapter 2

背景与相关工作

2.1 并行编程模型

PETSc 应用运行在跨节点分布内存、节点内非一致内存的系统上。编程模型决定哪些边界要由应用自己处理，哪些由硬件替它隐藏起来。

MPI 使用具有私有地址空间的独立进程。进程通过 MPI 标准定义的点对点操作和 collective 交换数据 [4]。每个 rank 拥有问题的一部分，通信显式表示远程依赖。MPI 也可在节点内通过共享内存传输，因此“纯 MPI”并不表示每条消息都经过网络。但它仍然是每个进程一个子域，仍要为每条消息做匹配，也仍要在每个 rank 里保留一份库状态。

OpenMP 采用共享内存 fork-join 模型。团队内线程访问同一地址空间，worksharing construct 划分循环迭代 [5]。进程内数据交换无需写成消息，但 runtime 仍要创建或唤醒线程、把工作分下去，并让它们在结束处互相等待。线程还会争抢 cache 容量和内存带宽。只有当并行区域里的工作足够多、能把这些成本赚回来时，线程化循环才会更快。

混合程序在每个节点放置一个或多个 MPI rank，并在每个 rank 内创建 OpenMP 团队。MPI 负责子域间通信，OpenMP 处理 rank 拥有的数据。rank 数与线程数的乘积给出使用的核心数，但乘积相同的布局并不等价。例如， 128×1 在一个 ARCHER2 节点上建立 128 个子域和 MPI endpoint； 32×4 只建立 32 个较大子域，每个子域使用四个线程。后者每个 rank 需要保存的求解器状态更少，需要跨越交换的子域边界也更少，代价是线程调度，以及共享同一 cache 和内存通道的线程之间的竞争。Table 2.1 总结三种模型的边界和主要成本。

Table 2.1: 三种执行模型在一个 ARCHER2 节点内的含义。混合模型同时承担 MPI 和 OpenMP 成本。

模型	内存	数据交换	主要成本
MPI	每个 rank 私有	显式消息	消息匹配、每 rank 求解器状态、每 rank 一个子域边界
OpenMP	团队内共享	共享地址	团队创建、同步、cache 与带宽竞争
混合	rank 内共享，rank 间私有	仅 rank 间使用消息	两类成本，由 rank-thread 配比权衡

PETSc 使用 MPI 完成分布式内存通信，因此本文比较纯 MPI 与 PETSc 的混合路径。其他编程模型不在本文范围内。

2.2 纯 MPI 的限制

纯 MPI 为每个核心分配一个 rank。它在节点内外使用相同的域分解和通信模型，每个 rank 执行单线程代码，因而放置较容易预测，也没有 OpenMP 调度成本。混合配置必须优于这一实测

基线。

强扩展固定全局问题并增加核心数，每核工作量随之下降。弱扩展让问题规模随核心数增长，以保持每核工作量，理想状态下单次迭代时间不变。前者测量固定问题能扩展多远。后者把局部工作量大致固定住，剩下要测的就是同时跑更多东西所付出的代价。Section 3.4 的筛选规则正是主实验成为弱扩展实验的原因，Section 4.1.1 给出具体链条。

强扩展时，每个 rank 的局部工作减少，但 latency-sensitive 操作不会同比缩小。Stencil 矩阵-向量乘仍需交换边界值，Krylov 点积仍需 collective reduction。局部子域缩小时，属于边界的部分变多、属于内部的部分变少，通信因而占据更大的迭代比例。rank 变多也意味着子域变多，而求解器和预条件器的状态每个 rank 都得自己存一份。

弱扩展并未消除通信，只是让通信的影响更容易单独看清。把每核未知量大致固定住，局部 kernel 的工作量就稳定下来，于是单次迭代时间的任何增长都来自随并发规模变化的那些量：邻居交换、参与全局归约的 rank 数，以及归约树的深度。Chapter 4 据此沿两条弱扩展链比较 rank 与线程。

以线程替代 rank 会减少 MPI endpoint、扩大 rank-local 子域，同时增加线程开销。Judge 在 ARCHER2 上发现，对通信较重的 CP2K 和 CASTEP，每 rank 两到四线程通常有效；测试的分子动力学程序则在接近扩展极限前收益很小 [6]。该平衡取决于具体应用。

2.3 混合 MPI+OpenMP

混合性能同时受 rank 数、每 rank 线程数和放置方式影响。固定核心数时，增加每 rank 线程数意味着 MPI endpoint 更少、每个 rank 内的子域更大，但每个 OpenMP 区域要协调的线程也更多。稀疏线性代数在这件事上尤其难办：它在内存里跳来跳去，每取一个值只做很少的运算，因此可能在所有核心都忙起来之前就把内存带宽用满。

Lange 等人使用 task-based OpenMP、通信重叠和线程负载均衡，使 PETSc 稀疏矩阵-向量乘的强扩展超过纯 MPI 极限 [7]。该 kernel 是为此专门写的，付出的远不止改一个线程数，其结果也不能用来预测当前 backend 的表现。

PETSc 3.22 加入配置项 `--with-openmp-kernels` [3]，在已标注的 kernel 中启用 OpenMP。它不会替代 MPI 分解，也不会让所有求解阶段都线程化。本文 build 同时使用 `--with-openmp=1` 和 `--with-openmp-kernels=1`。每 rank 单线程与混合运行使用同一 binary，从而把 runtime 布局与编译器、库差异分开。

混合运行扩展不佳时，原因可能是每线程工作量不足、某个阶段是串行的或被 MPI 主导、放置不当、cache 效应，或是共享内存上的竞争。它们中的任何一个都可能产生相同的 runtime 排序。Section 2.6 因此先用 profiling 把成本定位下来，再去争论它的成因。

2.4 ARCHER2 节点架构与 NUMA

一个 ARCHER2 计算节点包含两个 64 核 AMD EPYC 7742 (Rome) 处理器，共 128 个物理核心。每个 socket 配置四个 NUMA region，每节点八个；每个 region 有 16 个核心和本地内存通道。每个核心有私有 L1 和 L2 cache。四个核心组成一个 Core Complex (CCX)，共享 16 MB L3；两个 CCX 组成一个 Core Complex Die (CCD)。不同 CCX 不共享 L3 (Figure 2.1) [8]。

专用诊断任务在计算节点上确认了该 topology。lscpu 报告两个 64 核 socket、256 个逻辑 CPU、八个 NUMA node，以及分成 32 份的 512 MiB L3。numactl 显示每个 NUMA region 包含 16 个物理核心和约 32 GB 内存；region 内、同 socket 不同 region 和跨 socket 的距离分别为 10、12 和 32。sysfs 与 hwloc-ls 都显示一份 16 MB L3 位于四个物理核心之上。因此，实测边界为每 L3 四核、每 NUMA region 十六核。

NUMA 同时影响 latency 和带宽。Linux 通常把一页内存放在最先写入它的那个线程所属的 NUMA region。若一个线程初始化大数组，而整个线程团队随后从多个 region 去读它，这些流量就全落在拥有这些页面的那一个 region 的内存控制器上。ARCHER2 调优指南还指出，对带宽密集型代码，两个线程就已经可能接近一个 NUMA region 的内存带宽上限 [9]。更多线程并不保证更高带宽。

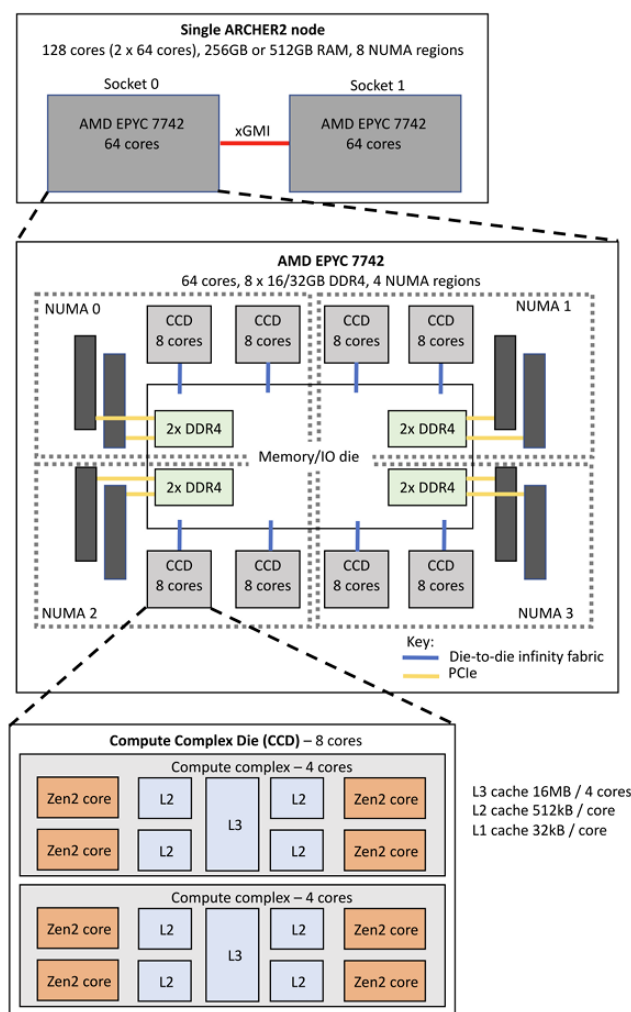


Figure 2.1: ARCHER2 计算节点的层次结构。每节点包含两个 64 核 AMD EPYC 7742 处理器、八个 16 核 NUMA region，以及各自共享 16 MB L3 cache 的四核 CCX。来源：ARCHER2 硬件文档 [8]。

Cache 层次划出的边界比 NUMA 更细：四个相邻核心共享一份 L3，而一个 16 核 NUMA region 包含四组这样的核心。在 Section 3.6 的 close binding 下，四种满节点布局相对这条边界的位置各不相同，见 Table 2.2。

Table 2.2: Close binding 下的四种满节点布局。它们均使用 128 个物理核心，但 rank 数和相对共享 L3 边界的位置不同。该表只描述放置，不断言性能机制。

布局	每 rank 核心数	每节点子域数	rank 与共享 L3 (CCX) 边界的关系
128 × 1	1	128	四个 rank 共享一个 CCX
64 × 2	2	64	两个 rank 共享一个 CCX
32 × 4	4	32	恰好占据一个 CCX
16 × 8	8	16	跨越两个 CCX

八线程跨越两组四核 L3，但仍待在同一个 16 核 NUMA region 内，而且本文测试的布局没有一个让 rank 跨越两个 NUMA region。所以这些布局之间的差别不可能来自跨 NUMA 边界，因为没有布局跨了。这排除了一种解释，但也没有证明 L3 或 OpenMP 的解释。Section 5.3 随后会指出采样线程等在哪里，但不确定为什么等。

2.5 PETSc 与 Krylov 求解器

PETSc 提供分布式向量和矩阵、Krylov 子空间求解器 (KSP)、预条件器 (PC) 以及本文使用的 logging 基础设施。PETSc 用 MPI 完成分布式内存通信, 并允许用户在运行时选择求解器组件 [1, 2]。这种组合方式便于实验, 但只有各布局执行可比的数值工作时, 并行布局比较才有意义。

正式实验求解有限差分 Laplacian, 其矩阵稀疏、对称且正定。共轭梯度法 (CG) 适用于这类线性系统 [10]。每次迭代都应用矩阵和预条件器、更新向量并计算内积。这些操作暴露两类扩展限制: 稀疏矩阵-向量乘与相邻子域交换 halo, 而内积和残差范数需要跨全部 rank 归约。Table 2.3 按此分类一次 CG step 的工作。

Table 2.3: 一次 GAMG 预条件 CG step 中的 PETSc event 和实测调用数。一个 step 对应一次预条件器应用; CG 会先预条件初始残差, 因此每次求解的 step 数比报告的迭代数多一。

操作	PETSc event	通信	每 step 调用数
矩阵-向量乘	MatMult	邻居	28.9
多重网格 restriction	MatMultTranspose	邻居	7.0
多重网格 prolongation	MatMultAdd	邻居	7.0
层级残差	MatResidual	邻居	7.0
上述操作的 halo exchange	VecScatterBegin	邻居	42.9
内积	VecTDot	全局	2.0
残差范数	VecNorm	全局	1.0
向量更新	VecAXPY, VecAYPX	无	44.7

多重网格层次在每个 level 交换 halo, 因此每 step 约有 43 次邻居交换和三次全局归约。两者随 rank 数的扩展方式不同, Section 5.4 分别测量它们。

预条件器改变系统频谱, 使 CG 用更少迭代达到指定 tolerance。代数多重网格 (AMG) 直接从矩阵构建层次结构。Smoothing 在每层减少高频误差, restriction、粗网格校正和 prolongation 处理局部 smoother 难以消除的误差分量 [11]。PETSc 的 GAMG (geometric-algebraic multigrid, PCGAMG) 无需几何 mesh 描述即可建立该结构 [1]; 本文没有向其提供几何信息。

GAMG 分为 setup 和 application。PCSetUp 构建 aggregate、transfer operator 和粗网格矩阵, 之后每次 PCApply 在 KSPSolve 中遍历层次结构。若多个线性系统使用同一 operator, 可只支付一次 setup, 因此首次求解时间与重复求解时间回答不同的性能问题。

预条件器还影响实验有效性。Block Jacobi 每个 MPI rank 建立一个 block, 布局变化时预条件器随之变化; AMG 没有这种直接耦合。Section 3.3 测量其影响, 并把正式比较固定在 GAMG 上。

2.6 性能分析工具

PETSc 的 `-log_view` 为每个 logging stage 报告时间、flop、消息、归约和命名 event [1]。MatMult、VecScatterEnd、VecTDot、PCSetUp 和 PCApply 等 event 把求解器操作与实测成本连接起来。它覆盖全部布局, 包括 128×1 基线, 因此提供跨布局可比量。

Linaro MAP 对 source-level call path、CPU、内存、向量化和 MPI metric 进行采样 [12]。它能区分核心正在执行 PETSc、等待 MPI, 还是停在 OpenMP runtime 路径中。Instrumentation 会带来较大且不均匀的开销, 因此 MAP 只解释 profiled 混合布局; 跨布局时间使用 `-log_view`。Profiled timing 不与未插桩的扩展结果混合。

2.7 小结与研究空缺

既有工作报告了通信密集型 ARCHER2 应用的混合收益 [6], 也展示了专用 PETSc 矩阵 kernel 的收益 [7], 但没有在配对二维、三维 stencil 问题上评估 PETSc 3.24 当前线程化 kernel backend 与 CG+GAMG, 也没有跨两种扩展模式系统比较 rank-thread 配比。本文研究该 backend 是否能在每节点固定 128 核时缩短稳态求解时间、哪种配比随并发增长表现最好, 以及哪些实测 event 跟踪该差异。实验固定总核心数、求解器和收敛条件, 改变布局、问题规模和节点数, 并把 runtime 与 `-log_view`、MAP 证据连接起来。

Chapter 3

实验平台与方法

3.1 硬件和软件环境

与安置有关的节点等级在 Section 2.4中描述。全体实验使用了物理核，同时多线程被禁用。软件堆栈为 PrgEnv-gnu 8.4.0, gcc 11.2.0, cray-mpich 8.1.27, 以及 PETSc 3.24.4。早期使用单节点勘探 arch-omp-opt。经验证的重复解决运动使用 arch-omp-libsci-rome-opt, 配置为线程 libsci_gnu_mp.so.5.0。LibSci is HPE Cray's tuned implementation of the Basic 线性代数子程序 (BLAS) 和 LAPACK, 密度的标准接口向量-向量、矩阵-向量和矩阵-矩阵操作;_mp 变体用 OpenMP 线程这些内核。PETSc 自己的内核和碰巧是达到密集的 BLAS。因此, 图书馆可能令人困惑, 而是速度来源: 其线程将来自相同的每条线程计数和和 PETSc 的一样被固定在相同的核心上并且会出现在同样的 OpenMP 运行时间中 Section 5.3 measures. That section reports how much of the solve it accounts for. 两者都启用了 PETSc 的 OpenMP 内核并使用 -O3. One thread per rank in the OpenMP-enabled build is the flat-MPI baseline.

3.2 基准问题与驱动程序

两个基准都在矩形域上求解 Poisson 方程 $-\nabla^2 u = f$, 边界条件为齐次 Dirichlet, 用二阶中心差分离散在均匀网格上。未知量是内部网格点, 因此网格边缘的一行会略去落在域外的邻居, 而网格间距被吸收进右端项。

二维基准基于 PETSc 的 KSP 教程 ex2(PETSc 源码树中的 src/ksp/ksp/tutorials/ex2.c [13])。在 $m \times n$ 网格上, 它把五点 stencil (对角 4, 非对角 -1) 组装成一个每行五个非零元的 $mn \times mn$ 稀疏矩阵。三维基准 3d_repeat 是为本项目编写的驱动程序, 它在 $m \times n \times p$ 网格上组装七点 stencil, 每行七个非零元, 其余结构相同。两个矩阵都是对称正定的, 两个驱动程序也都设置了 MAT_SYMMETRIC。

加入三维基准, 是因为一个子域中与本 rank 之外耦合的行所占比例在两个维度上并不相同, 而具体差多少由数据分解决定。两个驱动程序都按词典序给网格编号, $Ii = i + jm + kmn$, 并把局部规模交给 MatSetSizes 的 PETSC_DECIDE。PETSc 分给每个 rank 的是一段连续的行, 因此子域是这套编号里的一块薄板, 而不是立方体 (Figure 3.1)。记 L 为一个 rank 拥有的行数。一行通过 ± 1 、 $\pm m$ 以及三维中的 $\pm mn$ 与本 rank 之外耦合, 其中 ± 1 项最多只让两行跨越 rank 边界, 因此一个 rank 需要交换的 halo 大约是

$$2 \min(L, m) \quad (\text{二维}) \qquad 2 \min(L, mn) + 2 \min(L, m) \quad (\text{三维})$$

在二维, $\pm m$ 项的上限是 $2m$, 也就是 $2\sqrt{N}$, 相对 L 始终很小。在三维, 它的上限是 $2mn$; 一旦 L 降到 mn 以下, 每一行的两个 k 方向邻居都落在本 rank 之外, halo 达到 $2L$, 再也没法更小。所以三维问题与其说是 stencil 更难, 不如说是这种分解更早就没有内部可用了。Section 6.2 说明这给结果划出了什么范围。

两个问题都不使用解析形式的源项。它们都把精确解取为 $u = 1$ 并构造 $b = Au$, 因此每次运行都能报告 $\|x - u\|_2$, 作为端到端的已知解校验, 让求解出错时能在日志里看出来。网格是按

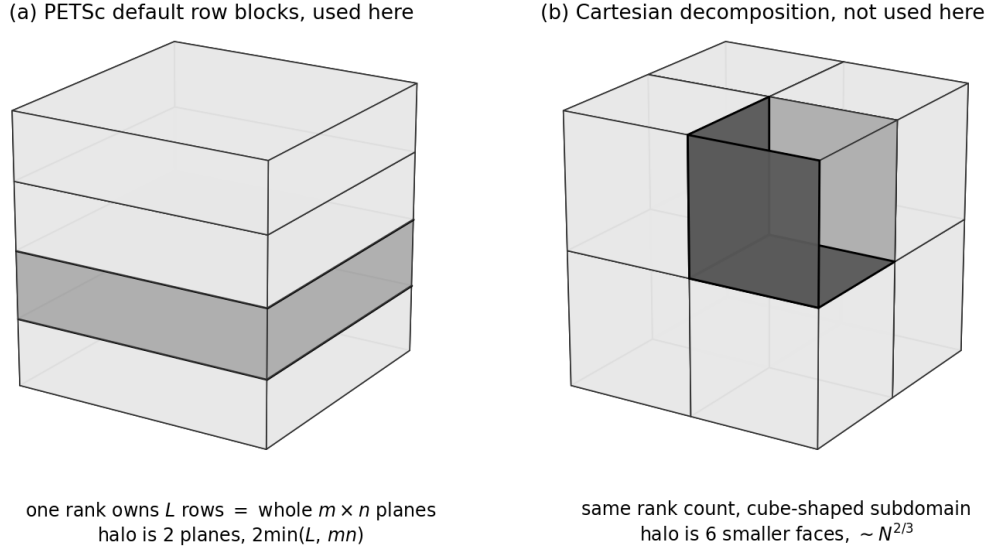


Figure 3.1: PETSc 默认的行分块如何决定子域的形状。词典序中连续的行使每个 rank 的子域成为若干完整 $m \times n$ 平面的堆叠，因此它的 halo 就是与 k 方向两个邻居共享的那两个平面；一旦 L 降到 mn 以下，halo 就不会低于 $2L$ 。笛卡尔分解则在每个方向上都切一刀，得到六个更小的面。本文所有运行都使用 (a)，(b) 是 Section 7.3 提出的替代方案。深色块是某一个 rank 的子域，深色表面是它需要交换的部分。

未知量配对的，而不是按网格边长：每个维度六种规模，从 5.0M 到 164.6M，每个二维规模都有一个误差在 0.5% 以内的三维搭档。Table 4.1 列出了它们以及各自运行的节点数。

两个基准都通过重复求解驱动程序运行，而不是只解一次。`ex2_repeat` 在一个专门的 `RepeatedSolves` 日志阶段里，把一次预热 `KSPSolve` 与 19 次计时求解分开，计时求解期间 `operator` 和预条件器都不变；`3d_repeat` 在三维采用同样的设计。每次计时求解前都会把初始猜测清零，否则上一次的解已经满足容差，下一次求解会在第 0 次迭代就返回。Section 5.1 给出这样做的理由和实测的成本拆分。就此而言，Chapter 4 中计时的每一次求解都是稳态求解。

3.3 求解器配置

默认 `ex2` 求解器是 GMRES；每个 MPI rank 对应一个 block-Jacobi block，block 内使用 ICC 分解。因此串行运行对整个矩阵分解，而每增加一个 rank 都会改变预条件器。在 2240×2240 及以上规模，所有运行都达到 PETSc 的 10,000 次迭代上限并报告 `DIVERGED_ITS`；最终误差范数为 10^0 – 10^3 数量级。任务还使用了非预期的 mesh-dependent tolerance，见 Section 3.8，因此不收敛是联合结果，不能单独归因。

正式运行使用

```
-ksp_type cg -pc_type gamg -ksp_rtol 1e-5 -ksp_atol 1e-10
```

离散 Laplacian 对称正定，因此使用 CG。椭圆问题上的多重网格收敛接近 mesh-independent，故选 GAMG。在 5.0–164.6 million 未知量范围内，最终求解二维使用 9–13 次迭代，三维使用 6–7 次。二维实测误差范数为 8.5×10^{-3} 至 1.34×10^{-1} ，三维为 6.0×10^{-3} 至 8.59×10^{-2} 。布局引起的小幅迭代数变化会影响接近的吞吐率比较，因此结果同时报告每秒求解方程数和每次 KSP 迭代时间。

Block Jacobi 还使预条件器成为 rank 分解的函数。Table 3.1 显示，在固定 rank 数时，迭代数不随线程数变化；跨 rank 数时则相差超过三倍。

该序列在重复运行中完全一致，并且不随 rank 数单调变化。横向读取只改变线程数，不改变收敛；纵向读取改变 block 分解，迭代数相差超过三倍。因此，不同 rank 数的布局对同一系统应用不同预条件器，wall-clock 时间无法回答哪种 rank-thread 布局最快。

Table 3.1: 1000×1000 网格上单节点默认求解器 sweep 的最终迭代数, 覆盖所有 $\text{ranks} \times \text{threads} \leq 128$ 组合。每行恒定, 说明迭代数只由 rank 数决定。128-rank 配置达到 10,000 次上限且未收敛。

MPI rank 数	每 rank 的 OpenMP 线程数							
	1	2	4	8	16	32	64	128
1	6700	6700	6700	6700	6700	6700	6700	6700
2	3715	3715	3715	3715	3715	3715	3715	
4	2825	2825	2825	2825	2825	2825		
8	4085	4085	4085	4085	4085			
16	8179	8179	8179	8179				
32	6460	6460	6460					
64	7181	7181						
128	10000							

GAMG 消除了“一 rank 一 block”的直接耦合。固定规模和节点数时, 二维布局最多相差两次迭代, 三维最多一次, 而不是数千次。每次 outer KSP 迭代时间控制了该收敛差异, 但不会使不同分解下的 GAMG 层次工作和通信完全一致。因此结果同时报告原始吞吐率和每次迭代时间。

GAMG 两个阶段的成本差异很大, 因此正式实验计时复用层次结构的重复求解, 而不是首次求解。Section 5.1 给出实测拆分和对应 driver。完整的默认求解器与 GAMG 比较需要更多配对且收敛的点, 本文不使用现有数据支持这种布局结论。

3.4 配置空间与筛选规则

三条规则定义正式配置空间。第一, 每 rank 线程数限制在 $\{1, 2, 4, 8\}$; 探索配置和正式配置都显示, 16 线程及以上的单次迭代成本明显更高 (Section 3.5)。这是实测 search bound, 不是 NUMA 或 cache 机制结论。

第二, 节点全部满载: $\text{ppn} \times \text{threads} = 128$, 得到 128×1 、 64×2 、 32×4 和 16×8 四种布局。相同节点数对应相同核心数, 因此比较隔离 rank-thread 配比与核心数。

第三, 规模为 N 、使用 C 个物理核心的配置必须满足 $25,000 \leq N/C \leq 100,000$ 。二维实验使用 2240^2 、 3160^2 、 4500^2 、 6400^2 、 9073^2 和 12828^2 六个网格, 即 5.02M 至 164.56M 未知量。筛选后在 1-32 个节点上保留 11 个规模-节点组合, 每个组合运行四种布局和三次正式重复。

3.5 限定线程数

Section 3.4 的第一条规则来自两组互补实验。探索 sweep 覆盖单节点上的全部 rank-thread 组合, 但使用会随 rank 数改变预条件器的默认求解器。之后的正式配置实验在 2、4、8 个节点上使用与 Chapter 4 完全相同的求解器、build 和 driver, 但每个配置只有一次运行。两者给出相同边界, 因此采用该限制。

3.5.1 探索性单节点 sweep

Sweep 在 1000×1000 网格 (100 万未知量) 上运行所有 $\text{ranks} \times \text{threads} \leq 128$ 组合。它早于求解器变更, 因此不能按 Section 3.3 对不同 rank 数的布局排序; 但固定 rank 数时预条件器也固定, 仍可用于限定配置空间。所有运行来自同一 build 和 campaign。下述两端共享一个基线: 单 rank、单线程, 6700 次迭代用时 245.7s。

Figure 3.2 比较使用节点 128 核的两种方式, 均从该基线出发。两条线都按每次 KSP 迭代归一化。纯线程线按构造固定预条件器; 纯 rank 线每个点都改变预条件器, 且 128-rank 配置未收敛。

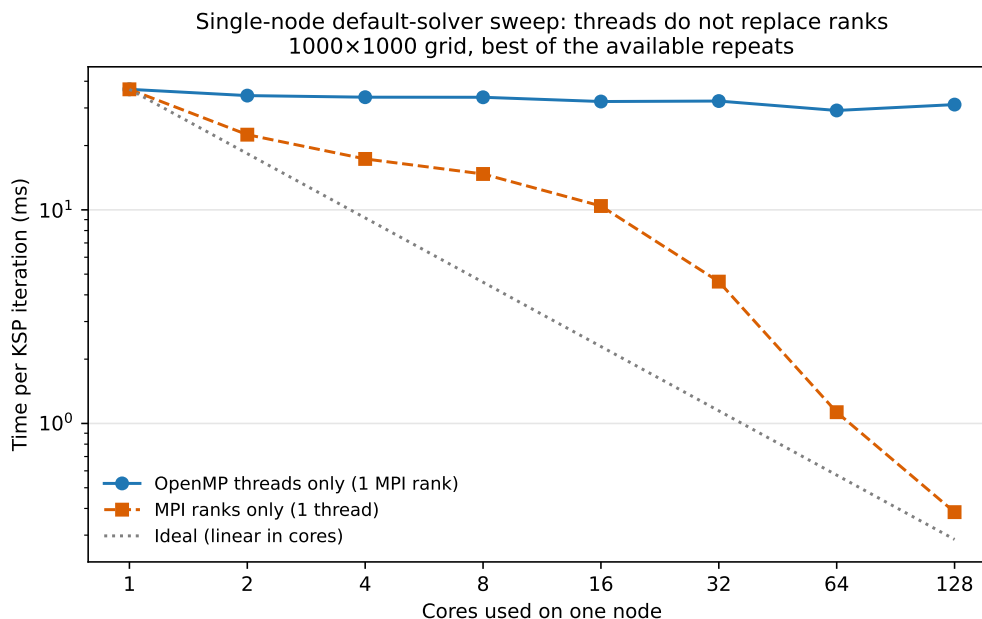


Figure 3.2: 1000 × 1000 网格上的单节点默认求解器 sweep。在单个 MPI rank 中增加 OpenMP 线程时，每次迭代成本几乎不变；保持每 rank 单线程并增加 MPI rank 时，成本下降两个数量级。两条线均从同一个单 rank、单线程运行开始。

纯 OpenMP 不缩放。每个线程计数时，一个等级都进行 6700 迭代，所以这些运行时间是可直接比较的，它们几乎不移动：245.7s 线程，225.5 在八，216.5 在三十二，195.6 在六十四，208.4 在 128。最佳案例是 1.26 线程的 64 因子，之后更多的线程花费时间。一次；128 线程返回 1.18 因子，用于 128 乘以核心。总的来说每次重迭成本的线程跨度为 1.26。超越排行它横跨 95.4。OpenMP 后端只平行执行路径的一部分，以及序列剩余部分将限制可实现的速度。Chapter 5 返回到此位置 profiler evidence.

同一基线的仅限 MPI 线在 64 排名上达到 8.1s，一个系数是：30。其等级计数不同，因此按 Section 3.3 显示，时钟数字是指示性的。而不是衡量平行效率。64 - 排名运行 *more* 比基线 (7181 相对于 6700) 重迭，因此重迭计数不能计算任何一个缺口。将这个问题分解到节点工作上，而线化则不。

由于固定核心的排位取代线程，每次迭代的成本也单调下降计数，如 Table 3.2 所示：趋势为 MPI rank 计数，无在任何特定线程计数时的阈值。

Table 3.2: Cost per KSP iteration for the eight fully populated single-node layouts (ranks × threads = 128) on a 1000 × 1000 grid. The trend is monotone in the rank count, with no threshold at any particular thread count. The 128 × 1 figure is measured over its non-converged 10,000 iterations, which still gives a valid per-iteration cost.

Layout	Iterations	Repeats	ms per iteration	Converged
1 × 128	6700	1	31.10	yes
2 × 64	3715	1	14.42	yes
4 × 32	2825	1	7.24	yes
8 × 16	4085	1	3.67	yes
16 × 8	8179	2	1.89	yes
32 × 4	6460	2	0.99	yes
64 × 2	7181	2	0.61	yes
128 × 1	10000	2	0.39	no

3.5.2 生产配置中的相同线程上限

上面的扫描不能按不同级别计数排列布局, 所以限制是重新测试生产结构, 然后将其应用于正式的实验 campaign。页:1 独立的多节点运动运行相同的 `arch-omp-libsci-rome-opt` 建设, 相同的 `ex2_repeat` driver, and the same CG+GAMG options as Chapter 4, at 2, 4 和 8 节点, 包括两个布局, 这一规则不包括: 8×16 和 4×32 。Table 3.3 gives the result. In all nine size-node cells the time per 迭代随每级线程单调增加, 被排除的布局不是边值: 8×16 成本 1.8 – 3.8 其单元格中最佳布局的两倍和 4×32 成本 3.0-7.4 倍。因此, 该规则将因数级处罚与被接纳的布局差别很小。

Table 3.3: KSP time per iteration for five full-node layouts in the production configuration. Each cell has one run and every row rises with threads per rank. The campaign has no 128×1 cell and bounds the search space rather than ranking the admitted layouts.

Scale	Nodes	Unknowns/core	ms per KSP iteration					Iterations
			64×2	32×4	16×8	8×16	4×32	
2m	2	7,810	1.75	2.25	3.52	6.08	11.82	9
2m	4	3,905	1.41	1.49	2.27	3.50	6.34	8–9
2m	8	1,953	1.30	1.30	1.64	2.32	3.88	8–9
4m	2	15,625	3.17	4.06	6.79	12.06	23.60	9–10
4m	4	7,813	2.13	2.52	3.80	6.46	12.39	8–9
4m	8	3,906	1.78	1.90	2.45	3.79	7.56	8–9
10m	2	39,006	11.79	13.25	20.01	30.68	61.51	9–10
10m	4	19,503	4.56	5.61	9.00	15.59	30.70	9–10
10m	8	9,752	3.10	3.28	5.05	8.65	17.03	9–10

一跑不能解决 0.5% 在 2m 上的 64×2 和 32×4 之间的差额。8 个节点. 正式的实验 campaign 对被录取的布局进行排名; 这项实验 campaign 只排除了 16 和 32 每级线程. 因此, 该研究保留 1、2、4 和 8。

3.6 作业配置与线程放置

每一等级及其线被钉在一组连成一体核上, 有:

```
OMP_PLACES=cores    OMP_PROC_BIND=close
srun --hint=nomultithread --distribution=block:block --exact
```

每个设置都是布局定义的一部分. `OMP_PLACES=cores` 造一个地方 (b) `OMP_PROC_BIND=close` 设置 MPI rank 相邻核心上的线程。 - - `hint=no multithread` 选择 128 物理核心比较 - - 分布 = 区块: 区块 连续核心, 而 - - 准确 将每项任务限制在所要求的范围内 `--cpus-per-task cores`. Under the measured topology of Section 2.4, a 32×4 级占据一个四核心共享-L3 域, 一个 16×8 级占据. 两个在两个 16-核心 NUMA 区域没有经过测试的布局设置一个等级. 固定夹针防止所研究的微小比例布局效果的模糊性与布局变化。

3.7 性能指标

主要时间是: `RepeatedSolves` stage. If this event has count $c = 19$, time t , and the grid has N 不明, 稳定状态吞吐量是

$$Q = \frac{Nc}{t}.$$

分析还报告了 $t/(cI)$, 其中 I 是 KSP 迭代的最终计数. 正式重复按中位数汇总; 胡须显示观察到的最小和最大值从三跑. 缩放量按布局的最小值计算节点数. 用于 n 节点的布局 ℓ ,

$$S\ell(n) = \frac{T_\ell(n_0)}{T_\ell(n)}, \quad E\ell(n) = \frac{S_\ell(n)}{n/n_0},$$

其中 n_0 是该问题大小最小的节点数。

两个百分比的公约保持不变。混合吞吐量 Q_h 和扁平-MPI 吞吐量 Q_m , 吞吐量增益为 $Q_h/Q_m - 1$. 他们的时代 t_h 和 t_m , 节省的时间是 $1 - t_h/t_m$. 他们描述的是相同的一对跑步, 但不是数字上的平等; 结果中的每个百分比都指它使用的公约。每次重复的时间是用于分离趋同-计数差异, 而不是主张相同的多网格工作. 三轮运行的最低和最高只能说明在这场运动中观察到的变化; 它们既不是信心间隔, 也不是意义测试。当两个观测范围, 文本将结果视为未解决, 而不是选择较小的中位数。当它们不重叠时, 结果支持本样本中的差异, 但仍然没有在测试配置之外建立通用命令。每一个直接的布局比较也有问题大小, 节点计数, 总物理核心, 确定解决方案和趋同容忍度; 仍可进行各种规模机制的比较即使它们有共同的锚点, 它们也会分开。

3.8 已废弃的实验 campaign

在正式研究之前, 早先的两项运动被放弃。他们的缺陷被保留在这里, 因为两个都看不到在结果的地块, 每一个动机现在检查由分析管道执行。

第一轮没有使用所要求的容忍。四位司机都继承一个依赖网格的亲戚 `KSPSetFromOptions` 之前的上游 `ex2` 的容忍称为: 2D 中的 `rtol = 10-2/((m + 1)(n + 1))` 和 3D 中的对应产品 `src/ex2_repeat.c:109` 和兄弟姐妹)。探索性的工作脚本取代了它双套选项, `--ksp_rtol 1e-5`, 而相邻的单套选项生效。800² 和 1000² 之间的迭代增加 39–62% 运动。Table 3.4 中的计算节点测试显示了原因。

Table 3.4: Relative tolerance actually used by `ex2` on a 100×100 grid, read from `-ksp_view`, for the three option styles. The double dash is silently ignored and the mesh-dependent default survives; `-options_left` names the unconsumed option explicitly.

Run	Option as passed	relative=	Iterations
G13-a	<code>--ksp_rtol 1e-5</code>	9.80×10^{-7}	84
G13-b	<code>-ksp_rtol 1e-5</code>	1×10^{-5}	65
G13-c	none	9.80×10^{-7}	84

双层选项从未被耗尽: 运行 G13-a 与无覆盖完全相同和 PETSC 的 `-options_left` 报告名称 `--ksp_rtol` 作为未使用. 有效容忍这些运动因此, 用网格收紧为 $1/N$, 在 6400^2 达到 2.4×10^{-10} . 那个在 Section 3.3 中报告的不一致性的后半部分: 要求的解决办法远比预期的准确, 附带先决条件, 而且随着 MPI rank 数的增加而减弱。探索性配置与生产不同 *both* 解析器和容忍度中的一个, 这就是为什么没有默认对 GAMG 的性能比较在本论文的任何地方。Section 6.2 重述限制。

第二轮在多个节点上使用 CG+GAMG, 但要求的布局没有运行. 每个 PETSc 日志都会在标题中显示进程数。这算数不同意 142 2D 日志和所有 161 3D 日志中的文件名, 通常报告要求进行 16、32、64 或 128 的一个过程。文件名记录意图, 不是执行, 所以两种不同标签的布局可能都有一个进程运行. 他们的 20–50 - 较慢的解析时间翻倍与该诊断一致。

因此, 从这些日志进行的任何布局比较都是无效的, 包括早先声称的 32×4 每个尺寸都最快. 这场运动还安排了一个以设置为主的解决方案: `PCSetUp` accounts for a median 69% of total time and is $3.3 \times$ the `KSPSolve` time. Seventy-two of 88 configurations have one run, so no observed range exists.

两种失败都产生了合理的产出, 而文件名则描述了意图而不是意图执行。因此, Section 3.9 中的审定合同检查所报告的构建、选项、进程计数和线程计数, 并在不匹配而不是静默的情况下失败丢一排。所替代的原木仍为原产地 (Appendix A)。探索论单节点扫描不受发射器缺陷的影响, 只在固定等级计数时使用在比较中, 其容忍性和先决条件是常见的。

3.9 可复现性

Chapter 4和 Chapter 5的每个结果都是由原始工作产生的由活动脚本输出; Appendix A 列出命令. 三项公约链条可审计。首先, 原`-log_view` 输出保留未修改, 而衍生的 CSV, 表格和数字是重新生成的, 而不是手工编辑的. 第二, 解析器信任日志内容而不是文件名。他们检查报告的建筑, 过程和线程计数 PETSc 已看到、解析和事件计数, 以及链接库。出现不匹配现象而不是被丢弃。第三, 脚本显示音素中使用的属性, 所以一个数据使索赔无效的变更也使管道失效。

例如, Table 3.1 的生成器拒绝写入表格除非迭代计数在每一个固定的等级计数中都是连续的。三十个单位测试包括解析和运动完整性。限制仍然明确:3 个重复支持一个观察到的范围, 但并非一个重大索赔, 而且暴露量份额在 Section 5.4 are indicators rather than an additive time budget.

MAP 分析遵循的是同一合同. 它读取每个压缩 `.map` XML 直接文件, 验证所记录的 `srun` 版式, 拒绝其框架的配置文件并坚持文本中使用的订单。这样可以避免许可的 GUI 导出步骤, 并保留配置文件作为源证据。

Chapter 4

2D 与 3D 问题

4.1 实验设置

每个维度各有两场满节点实验，平台、build、驱动程序、求解器、放置方式和测量约定都取自 Chapter 3。每个节点都占满 128 个物理核心。两场实验只在扩展场景上不同：弱扩展让问题规模随节点数增长，以保持局部工作量大致不变；强扩展则固定一个问题，用来看它在哪里饱和。Table 4.1 概括了两种设计。

Table 4.1: Grids and node counts. Weak-scaling points pass the unknowns-per-core filter of Section 3.4; the fixed-size campaign runs the 20m problem over all nodes.

Scale	2D grid	2D unknowns	3D grid	3D unknowns	Weak-scaling nodes	Fixed-size nodes
5m	2240 ²	5,017,600	171 ³	5,000,211	1	—
10m	3160 ²	9,985,600	215 ³	9,938,375	1, 2	—
20m	4500 ²	20,250,000	273 ³	20,346,417	2, 4	1, 2, 4, 8, 16, 32
41m	6400 ²	40,960,000	345 ³	41,063,625	4, 8	—
82m	9073 ²	82,319,329	435 ³	82,312,875	8, 16	—
165m	12828 ²	164,557,584	548 ³	164,566,592	16, 32	—

弱扩展实验包含 11 个规模-节点组合，固定规模实验包含 6 个。每个组合都跑四种布局、各重复三次。在进入中位数之前，每份日志都要核对它报告的 build、实际的进程数和线程数、20 次收敛求解、19 个计时的 **RepeatedSolves** event、预条件器复用，以及链接的 LibSci 路径。全部 280 份弱扩展日志和 144 份被接受的固定规模日志都通过了核对。检索到的 140 行三维源 CSV 也与独立解析出的日志一一对应。每个维度另有 8 份更早的 20m 试运行日志，作为出处保留，但不进入任何中位数和图表。

三次重复的最大最小-最大波动，弱扩展下二维为 4.71%、三维为 3.83%，求解时间更短的固定规模实验下则分别为 10.77% 和 11.09%。小于这个幅度的布局差距按平局报告。

4.1.1 弱扩展链

筛选条件 $25,000 \leq N/C \leq 100,000$ 把问题规模和节点数绑在一起，每个规模只剩下一两个节点数。因此按规模分面的图，每一面最多只有两个点。这 11 个通过筛选的组合改为排成两条每核工作量近似恒定的链 (Table 4.2)，从而在不改动底层测量的前提下，得到五点或六点的并发成本视图。

六点链和五点链都落在各自标称带的 15% 以内。它们按弱扩展来读，而固定规模下相邻的节点数仍是局部的强扩展观测。理想的弱扩展应当保持每次迭代时间不变，沿某一行的增长就是局部工作量大致固定时的并发成本。两种读法共用同一批数据，但回答的是不同的问题，不会被汇总成一个新的统计量。

Table 4.2: The 11 admitted pairs arranged as two weak-scaling chains with near-constant unknowns per core.

Band	Size at node count					
	1	2	4	8	16	32
~40k unknowns/core	5m	10m	20m	41m	82m	165m
~80k unknowns/core	10m	20m	41m	82m	165m	—

4.1.2 强缩放设计

相邻节点数的测量，每个规模只给出一步强扩展，看不出曲线在哪里转折。因此配对的扩展实验固定 20m 问题，对每种布局都跑 {1, 2, 4, 8, 16, 32} 个节点，求解器和重复求解的约定都不变。在 32 个节点上每核只剩约 4.9 千个未知量，远低于 Section 3.4 准入的每核 2.5 万到 10 万。这个有意过度分解的端点是用来探测饱和的，不应当被读作高效的运行区间。

2 节点和 4 节点这两个锚点重复了早先弱扩展实验里采集过的规模-节点组合，用来检验两次会话之间有没有偏移。二维的中位数相差最多 2.2%，八个区间里有六个重叠。三维的重跑慢了 0.3–3.1%，八个区间没有一个重叠，因此两次会话分开报告，不做合并。

三维的 1、2、32 节点点还提交过两次。分析取的是较新的、三次重复齐全的那一批，另外 36 份更早的日志作为被取代的出处保留。两批数据在 1 节点和 2 节点上相差不超过 0.78%，在 32 节点上相差 -4.7% 到 +5.1%，而那里的求解时间只有几十毫秒。两批都在 32 节点选出 16×8 ，分别比纯 MPI 快 $3.81\times$ 和 $4.00\times$ 。因此关于饱和的结论不依赖于用哪一批，但两批并没有合并成一个六次重复的中位数。

4.2 每核工作量恒定时的弱扩展

两个维度使用同样的 11 个组合、四种布局和三次重复。

4.2.1 2D

沿两条链，吞吐量都接近线性上升，而在这个尺度上布局之间只差几个百分点 (Figure B.1)。Figure 4.1 则把每次 KSP 迭代的时间除以同一组合下纯 MPI 的对应值，消去规模和 9–13 次 GAMG 迭代的影响。

六条比值序列中有四条是单调的，也就是说随节点数每上一步，混合布局相对纯 MPI 都在拉开差距。沿 ~40k 链， 64×2 的比值从单节点的 0.957 降到 16 节点的 0.872，到 32 节点小幅回升至 0.885， 32×4 则从 1.096 降到 0.937。沿 ~80k 链，三种混合布局的比值每一步都在下降。因此 64×2 在全部 11 个组合上每次迭代都更快，而 32×4 跨过了基线：单节点贵 9.6%，8 节点持平 (1.003)，32 节点便宜 6.3%。所以小节点数下的排名不能原样推广。

原始吞吐量还包含收敛次数的差异。 64×2 在 11 个组合中的 8 个领先。三次纯 MPI 领先里有两次伴随少一次 GAMG 迭代，第三次是 0.4% 的平局，区间互相重叠。在 32 节点 165m 上， 64×2 达到 794.8 百万方程/秒，比纯 MPI 高 13.1%。它的最大值出现在 16 节点 82m 上：吞吐量增益 14.7%，时间节省 12.8%。

16×8 在全部 11 个组合上的吞吐量都是最低的，不过它的差距从 46.2% 收窄到 22.7%。

Figure 4.2 给出对应的每次迭代绝对时间。

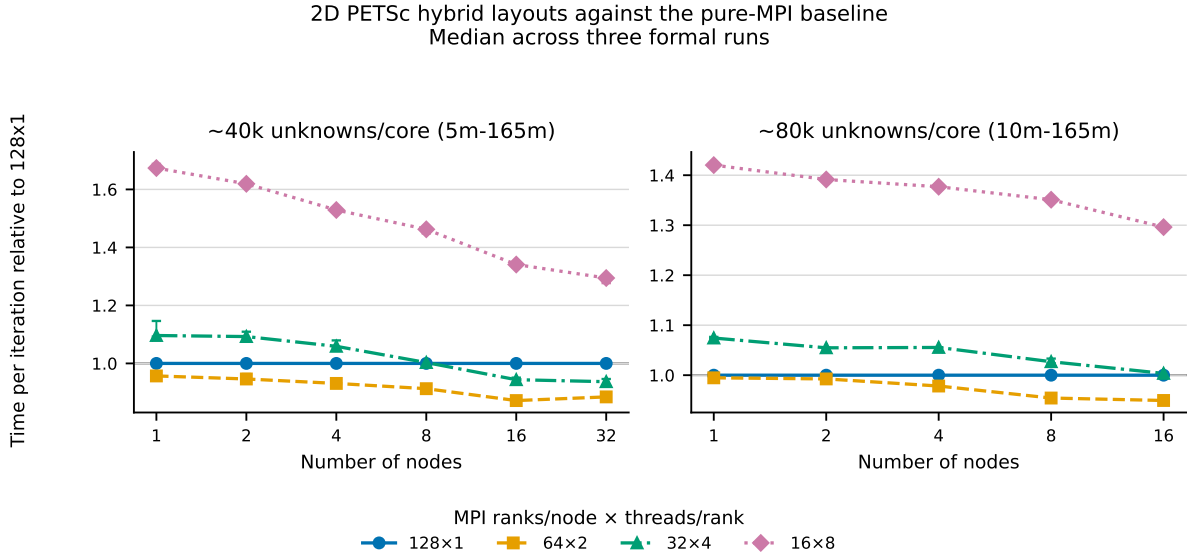


Figure 4.1: 2D time per KSP iteration relative to the 128×1 baseline at the same size–node pair; values below 1 mean the hybrid layout is faster. Whiskers propagate the observed range of three formal runs.

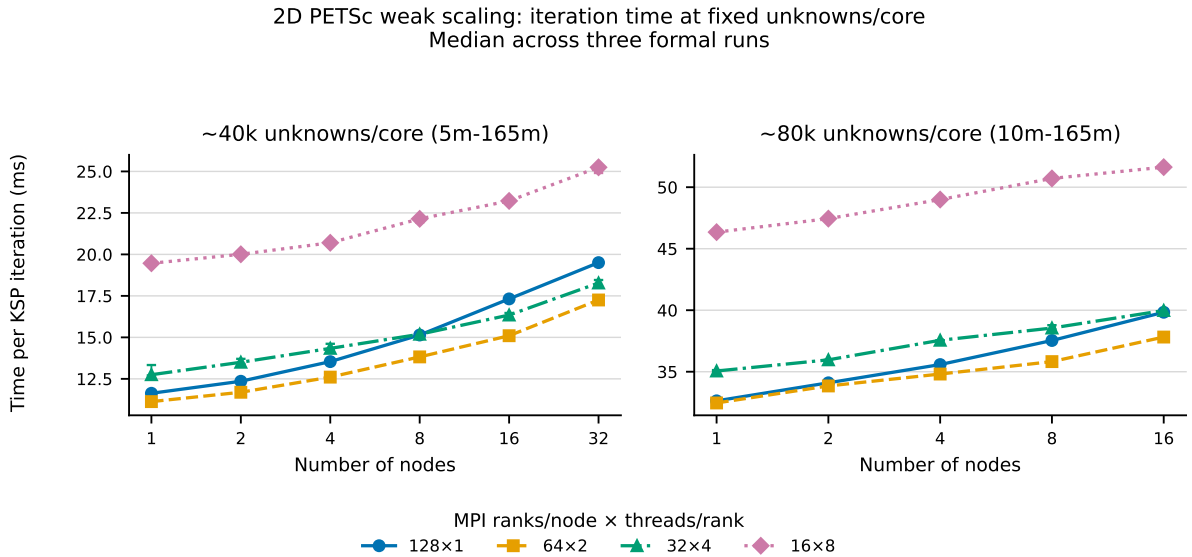


Figure 4.2: Median 2D time per KSP iteration along the two unknowns/core chains. A horizontal line would be ideal weak scaling.

4.2.2 3D

三维实验采用同样的设计，KSP 迭代次数为六到七次。和二维一样，吞吐量接近线性，而把时间按每次迭代归一之后，各布局更容易分辨 (Figures 4.3 and B.2)。

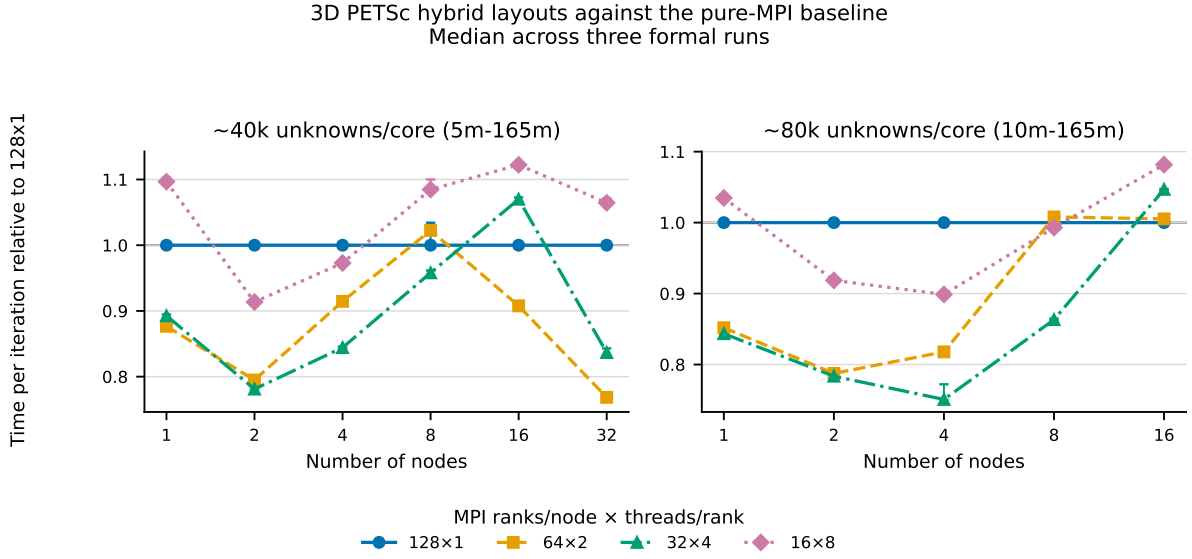


Figure 4.3: 3D time per KSP iteration relative to the 128×1 baseline at the same size-node pair. Unlike the 2D case in Figure 4.1, no series is monotone.

三维的优势更大，但次序更乱。在四节点 41m 上， 32×4 给出最大的时间节省 (24.9%) 和最大的吞吐量增益 (33.2%)，二维对应的最大值则是 12.8% 和 14.7%。这两个百分比不同，是因为一个比的是时间、另一个比的是速率。33 个混合测量点中有 22 个每次迭代快于纯 MPI，二维则是 33 个中的 13 个。

六条三维比值序列全都不是单调的。在 $\sim 80k$ 链上，混合布局在 2 节点和 4 节点最强， 32×4 达到 0.784 和 0.751，到 16 节点则回到或越过持平线，那里纯 MPI 每次迭代最快。在 $\sim 40k$ 链上， 64×2 从 8 节点的 1.022 回到 32 节点的 0.768。因此三维的优势比二维大，但在不同节点数之间更难迁移。

按原始吞吐量选， 32×4 拿下 6 个组合， 64×2 4 个，纯 MPI 1 个。在 32 节点 165m 上， 64×2 达到 510.4 百万方程/秒，比纯 MPI 高 30.1%。 16×8 一个组合都没赢，但在其中 5 个上胜过纯 MPI。

Figure 4.4 给出每次迭代的绝对时间。

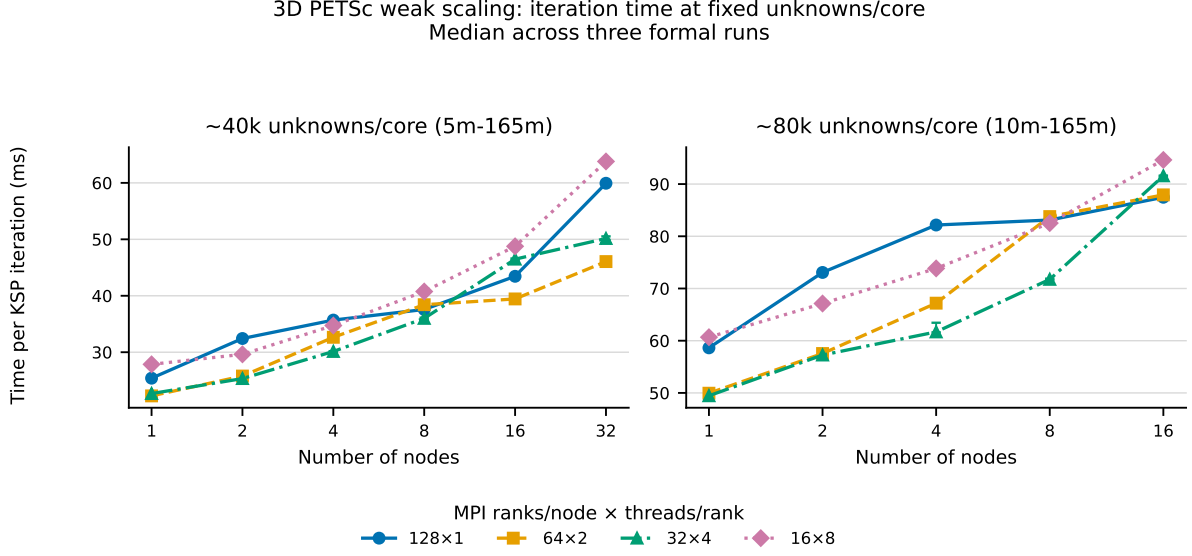


Figure 4.4: Median 3D time per KSP iteration along the two unknowns/core chains.

4.2.3 布局排名与效率

Table 4.3 给出每个组合下最快的布局及其相对纯 MPI 的优势。

Table 4.3: Fastest layout by median throughput over three repeats, its margin over flat MPI, and time per iteration. The 2D 41m/8-node row is a tie because ranges overlap.

Scale	Nodes	Cores	Best layout	M eq/s	vs. MPI	ms/iteration
<i>2D, five-point stencil</i>						
5m	1	128	128 × 1	47.9	0.0%	11.63
10m	1	128	128 × 1	34.0	0.0%	32.63
10m	2	256	64 × 2	94.9	5.7%	11.69
20m	2	256	64 × 2	59.8	0.8%	33.84
20m	4	512	64 × 2	160.7	7.4%	12.60
41m	4	512	64 × 2	107.0	2.2%	34.81
41m	8	1024	128 × 1	270.5	0.0%	15.14
82m	8	1024	64 × 2	208.9	4.8%	35.82
82m	16	2048	64 × 2	495.6	14.7%	15.10
165m	16	2048	64 × 2	362.6	5.3%	37.82
165m	32	4096	64 × 2	794.8	13.1%	17.25
<i>3D, seven-point stencil</i>						
5m	1	128	64 × 2	37.4	14.1%	22.26
10m	1	128	32 × 4	28.7	1.6%	49.45
10m	2	256	64 × 2	64.3	25.8%	25.78
20m	2	256	32 × 4	50.8	27.6%	57.26
20m	4	512	32 × 4	96.5	1.5%	30.13
41m	4	512	32 × 4	95.1	33.2%	61.68
41m	8	1024	32 × 4	163.0	4.4%	35.99
82m	8	1024	32 × 4	163.9	15.8%	71.75
82m	16	2048	64 × 2	298.2	10.2%	39.44
165m	16	2048	128 × 1	268.8	0.0%	87.47
165m	32	4096	64 × 2	510.4	30.1%	46.06

二维的获胜布局在 11 行里换了两次，三维换了五次。这与二维比值有序、三维比值随规模变化的趋势一致。

以每种布局自己的单节点时间为基线，得到 Table 4.4 中的弱扩展效率。

Table 4.4: Weak-scaling efficiency (%) at the largest node count of each chain, relative to the same layout’s one-node time. Each layout is measured against itself, so this quantifies degradation, not absolute speed.

Case	Chain and node count	Layout			
		128×1	64×2	32×4	16×8
2D	~40k, 32 nodes	59.6	64.5	69.8	77.1
2D	~80k, 16 nodes	81.9	85.8	87.7	89.8
3D	~40k, 32 nodes	42.4	48.3	45.2	43.7
3D	~80k, 16 nodes	67.0	56.8	54.0	64.1

在二维，两条链上的效率都随每 rank 线程数上升。这与比值曲线下降的趋势一致，但并不代表绝对速度的排名： 16×8 效率最高，而它在每一个点上的时间都是最差的。以各布局自身基线来衡量的指标，会偏袒基线本来就差的布局。

三维的效率没有按线程数排列： 64×2 在 ~40k 链领先，纯 MPI 在 ~80k 链领先。二维的模式不能迁移过来。

4.3 固定问题规模下的强扩展

固定规模实验考察增加节点何时不再缩短运行时间。两个维度均使用相同的 6 个节点数和 4 种布局。

4.3.1 2D

5 次相邻节点数翻倍带来 2.00–3.08 的加速。超线性步骤在 3 次重复中均出现，但其原因尚未确定。

所有 2D 固定规模曲线到 32 节点仍在改善 (Figure 4.5)。最快与最慢布局的差距由单节点的 $1.47\times$ 扩大到 16 节点的 $1.81\times$ ，随后在 32 节点缩小到 $1.48\times$ 。 64×2 在节点数 $\{2, 4, 8, 32\}$ 时最快，纯 MPI 在 $\{1, 16\}$ 时最快。 64×2 在 8 节点的累积效率达到 191%；缓存容量和 GAMG 层次变化仍只是未经验证的解释。

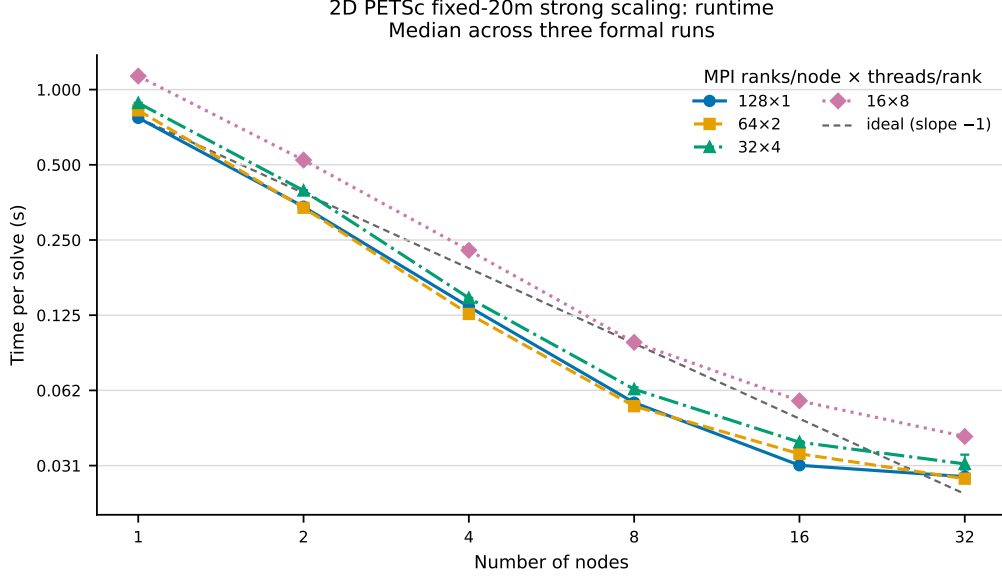


Figure 4.5: 2D time per solve at a fixed 20.25M unknowns, one to 32 nodes. Both axes are logarithmic, so ideal strong scaling is a straight line of slope -1 . Whiskers show the observed range of three formal runs.

4.3.2 3D

5 次相邻节点数翻倍带来 1.46–2.39 的加速，没有超线性步骤。在 165m 规模下， 64×2 从 16 到 32 节点获得 $1.91 \times$ 加速，而纯 MPI 为 $1.46 \times$ 。

3 种 3D 布局在测量范围内出现运行时间回升 (Figure 4.6)。纯 MPI 在 8 节点达到最低点， 64×2 和 32×4 在 16 节点达到最低点；只有 16×8 到 32 节点仍继续改善。此时其 0.0681s 求解时间在相同的 4096 核上比纯 MPI 快 $3.81 \times$ ，并比 32×4 快 10%。互不重叠的观测范围支持该端点存在布局差异，但实验尚未定位 16×8 更后的饱和点。

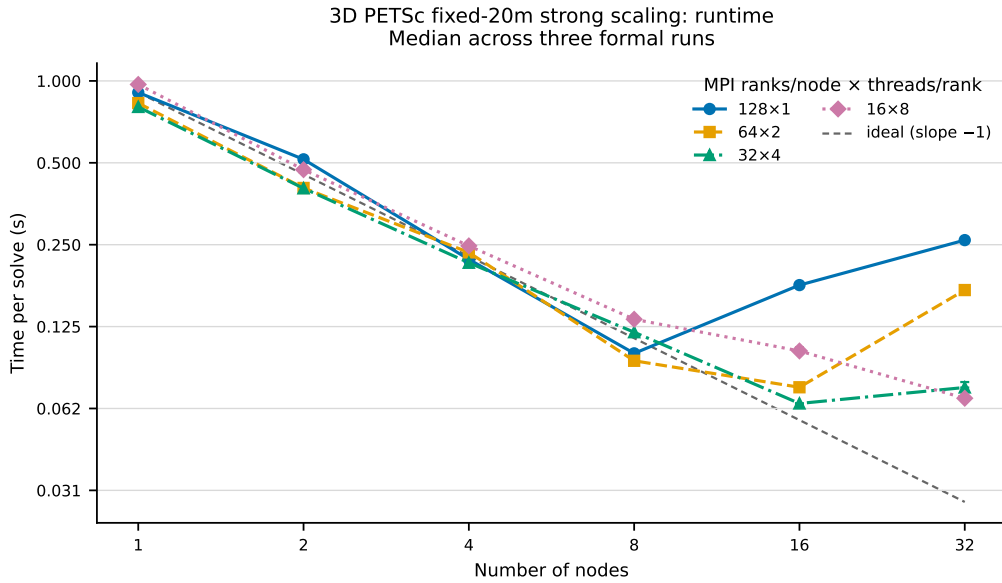


Figure 4.6: 3D time per solve at a fixed 20.35M unknowns. Flat MPI bottoms at eight nodes, 64×2 and 32×4 at 16; only 16×8 still improves at 32.

Figure 4.7 以各布局自身的单节点时间为基线，因此比较的是扩展行为而非绝对速度；Sec-

tion 4.3.3 同时报告二者。2D 曲线没有回升，其效率列于 Table 4.6。

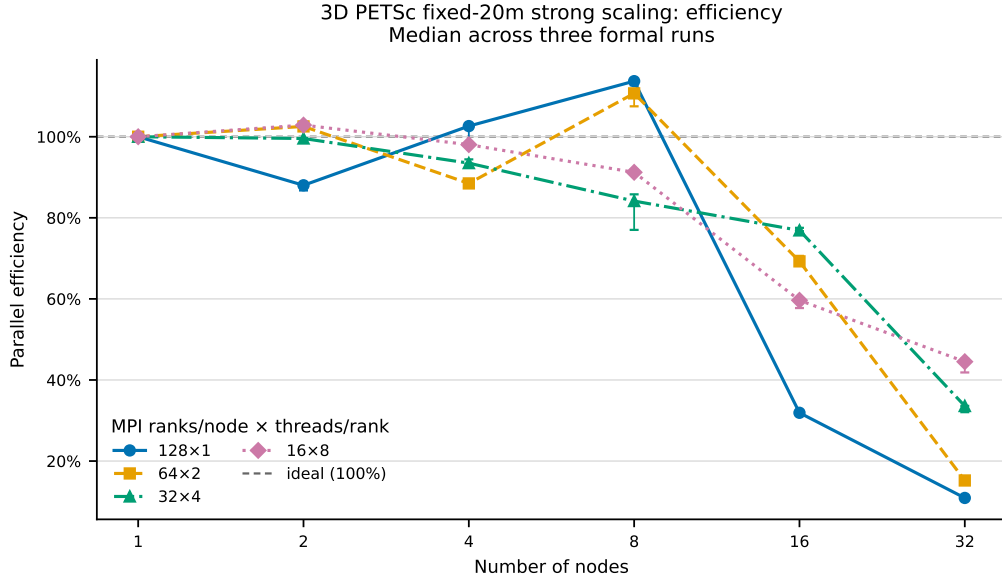


Figure 4.7: 3D parallel efficiency at a fixed 20.35M unknowns, each layout against its own one-node time. Because the baselines differ, this figure ranks scaling behaviour and not speed; flat MPI falls to 10.9% at 32 nodes while 16×8 retains 44.5%.

4.3.3 端点比较

Table 4.5 给出每个节点数下的最快布局及其相对纯 MPI 的优势。2D 优势始终低于 6.9%；3D 优势在 32 节点达到 281.3%。

Table 4.5: Fastest layout for each fixed 20M problem, by three-run median time per solve, and its advantage over flat MPI.

Nodes	Cores	2D, 4500 ²			3D, 273 ³		
		Best	s/solve	vs. MPI	Best	s/solve	vs. MPI
1	128	128×1	0.7704	0.0%	32×4	0.8022	13.0%
2	256	64×2	0.3365	1.0%	32×4	0.4028	27.8%
4	512	64×2	0.1265	6.9%	32×4	0.2145	2.9%
8	1024	64×2	0.0541	3.3%	64×2	0.0936	6.5%
16	2048	128×1	0.0313	0.0%	32×4	0.0652	172.4%
32	4096	64×2	0.0277	1.9%	16×8	0.0681	281.3%

在 32 节点, 2D 速度集群在 27.3-29.8, 因为每个布局都使用自己的一个节点虽然终点运行时间因 45 % (Table 4.6) 而异。内 3D 速度和运行时间排名一致。

Table 4.6: The 32-node endpoint of the fixed-size curves. Speedup and cumulative efficiency are measured against the same layout’s one-node time, so they reward a poor baseline; the doubling efficiency is the 16-to-32-node step alone. Time per solve is the only column that compares layouts directly.

Case	Layout	s/solve	Speedup	Efficiency	16→32 step
2D	128×1	0.0282	27.30	85.3%	55.5%
2D	64×2	0.0277	29.83	93.2%	62.9%
2D	32×4	0.0317	27.85	87.0%	61.1%
2D	16×8	0.0409	27.71	86.6%	69.4%
3D	128×1	0.2599	3.49	10.9%	34.2%
3D	64×2	0.1703	4.87	15.2%	22.0%
3D	32×4	0.0747	10.75	33.6%	43.7%
3D	16×8	0.0681	14.24	44.5%	74.6%

4.4 综合解释

Table 4.7 概括了两趋势形状的差别。三维在 16 和 32 节点上相对纯 MPI 的差距超过了实测的运行间波动；二维固定规模下 1–7% 的优势没有超过，因此单个二维排名只能作为参考。

Table 4.7: Cross-dimensional weak- and strong-scaling summary. Throughput gain and time saving use rate and time conventions respectively. A layout turns upward when its time after its measured minimum increases.

	2D	3D
<i>Constant work per core, 11 size-node pairs</i>		
Layout winning most points	64×2 (8 of 11)	32×4 (6 of 11)
Hybrid points faster per iteration	13 of 33	22 of 33
Largest throughput gain	14.7%	33.2%
Largest per-iteration saving	12.8%	24.9%
Ratio series monotone in nodes	4 of 6	0 of 6
Weak efficiency rises with threads	yes	no
<i>Fixed 20M problem, one to 32 nodes</i>		
Layouts whose time turns upward	none of 4	3 of 4
First to turn upward	—	128×1 , at 8 nodes
Speedup on the last doubling	1.11–1.39×	0.44–1.49×
Fastest at 32 nodes	64×2	16×8
Its lead over flat MPI	1.02×	3.81×
Fastest-to-slowest spread at 32 nodes	1.48×	3.81×

二维六条比值序列中有四条单调，弱扩展效率随线程数上升。三维没有一条序列单调，最佳布局随规模和节点数变化。扩展场景同样重要：每核工作量恒定时布局差异在百分点量级，而固定的三维问题在 32 节点上差距达到 3.81×，因为各布局是在不同的实测点上饱和的。

由于两种 stencil 每个方程的工作量不同，跨维度的比较使用的是布局次序、相对纯 MPI 的优势和趋势形状，而不是绝对吞吐量。

本章没有指出二维混合比值持续改善、或 16×8 绝对性能很差背后的机制。Chapter 5 检验事件暴露量和 OpenMP 等待是否与这些观测对应，Chapter 6 把合起来的证据转成建议。

因此，一条布局建议必须写明它所依据的扩展场景。 16×8 在两个维度上按每核工作量恒定来看都排最后，但在固定的三维问题上、在 4096 核时却排第一；这两组测量回答的是不同的问题。

Chapter 5

性能剖析与瓶颈分析

5.1 为何使用重复求解驱动程序

在 20.25M 问题上用 16×8 做一次单次求解时, `-log_view` 给出 `PCSetUp` 占 45% 的时间、`KSPSolve` 占 10%。`ex2_repeat` 把求解放在一个 `RepeatedSolves` 日志阶段里跑, 每次求解前把初始猜测清零, 并复用预条件器(`-ksp_reuse_preconditioner` 与 `-pc_gamg_reuse_interpolation true`)。跑 20 次求解时, 该阶段占了 69% 的时间和 93% 的浮点运算; 在阶段内部 `KSPSolve` 占 100% 的时间; 19 次重复每次都是 11 轮迭代。修正后的扩展实验采用同样的预热与复用设计, 因此 Chapter 4 比较的是稳态求解, 而不是首次访问和层次结构构建的成本。

5.2 分析方法

方法是把 MAP 25.0.4 的时间线与 `-log_view` 的 %T (占阶段时间的比例) 和 %F (占阶段浮点运算的比例) 结合起来。某个 event 的 %T 若超过它的 %F, 超出的部分就花在通信、同步或内存访问上。Table 5.1 列出 20.25M、 16×8 下 `RepeatedSolves` 阶段的单点数据。

Table 5.1: Per-event time and flop shares, `RepeatedSolves` stage, 20.25M, 16×8 .

Event	%T	%F	GF/s
MatMult	39	55	30.5
MatMultTranspose	18	6	6.7
VecTDot+VecNorm	8	6	—
PCApply (agg.)	82	82	21.6

`MatMultTranspose` (GAMG 的 restriction) 的 %T 是它 %F 的三倍, 速率比 `MatMult` 低 $4.6\times$ 。`VecTDot` 和 `VecNorm` 是 CG 的全局归约。

两种工具回答的是不同的问题。未插桩的 `-log_view` 运行覆盖纯 MPI 和每一种混合布局, 因此它提供布局之间的比较, 并指出具体 PETSc event 上的暴露量。MAP 对调用树采样, 把 core-time 归给 PETSc、MPI、OpenMP 和各个库, 但它启动不了 128 rank 的基线, 而且会扰动运行时间。所以它用来定位混合布局的核心在忙什么或在等什么, 而不是用来提供扩展时间。两者的输出都不是对另一方的可加分解: PETSc event 是嵌套的, 且按 rank 上的最大值报告, 而 MAP 的比例是对采样到的 core-time 做划分。两者次序一致, 只是事件核算层面上的旁证, 不等于可以把两边的量相加, 也不等于可以推出硬件成因。

5.3 MAP profiling 结果

两场 MAP 实验剖析的是同一个 20.25M 二维问题:

1. 一组基线跑在 `arch-omp-opt` 上, 节点数为 2、8、16;

2. 另一组与之对应，跑在生产 build `arch-omp-libsci-rome-opt` 上，节点数为 2 和 8。

两组都没有 128×1 那一格，因为 MAP 在本系统上无法每节点启动 128 个 MPI rank，所以纯 MPI 只有 `-log_view` 这一种证据。12 份 profile 中这里用了 11 份，16 节点的 16×8 基线 profile 因其自身内容被剔除：MAP 在其中没有解析出任何库符号，这份文件不含成本信息。

求解窗口范围只保留这样的采样：有核心处于 `KSPSolve` 内部，且没有任何核心处于 `PCSetUp` 内部，这样就仅凭 profile 近似出重复求解阶段。它是一个时间窗口，不是 PETSc 的 `RepeatedSolves` 日志阶段，两者不可当作同一个量引用。Table 5.2 给出由此得到的比例，Figure 5.1 将其画出。

Table 5.2: Share of total core-time inside the solve window, by the library owning the executing frame, for the eleven usable MAP profiles at 20.25M unknowns in 2D. OpenMP is the GNU runtime’s spin and wait paths, that is threads with no work. MAP instrumentation inflates these runs, so the ordering between layouts is the result. The absolute values are not transferable to the uninstrumented campaigns.

Build	Nodes	Layout	PETSc	OpenMP	MPI	LibSci
LibSci	2	64×2	31.9%	36.5%	29.9%	1.0%
LibSci	2	32×4	51.6%	40.1%	5.8%	1.7%
LibSci	2	16×8	42.2%	53.5%	1.4%	2.1%
LibSci	8	64×2	4.5%	48.3%	46.7%	0.2%
LibSci	8	32×4	7.5%	69.3%	22.6%	0.3%
LibSci	8	16×8	15.6%	76.5%	6.2%	1.0%
Baseline	2	32×4	51.7%	40.8%	6.7%	0.0%
Baseline	2	16×8	40.2%	57.8%	1.4%	0.0%
Baseline	8	32×4	8.8%	68.6%	22.2%	0.0%
Baseline	8	16×8	16.0%	77.5%	6.0%	0.0%
Baseline	16	32×4	1.2%	69.8%	29.0%	0.0%

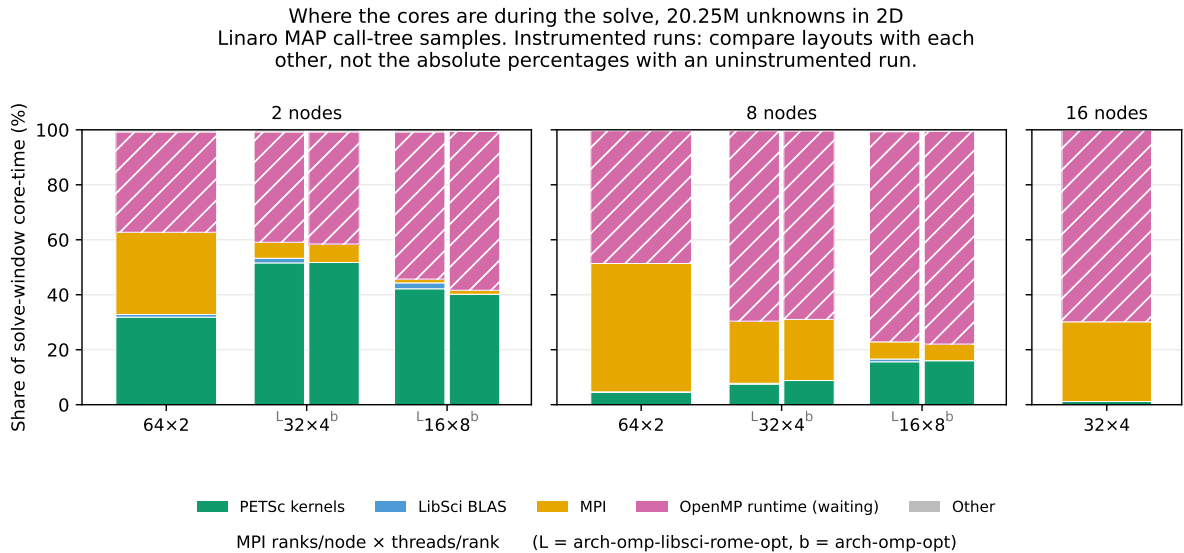


Figure 5.1: Where the cores are during the solve, from the MAP call trees. Within each panel the OpenMP waiting share grows and the MPI share shrinks as threads replace ranks, and both effects strengthen with node count. Paired bars are the two PETSc builds at the same configuration.

两类成本此消彼长。MPI 的比例随线程替代 rank 而下降：在 8 个节点上， 64×2 、 32×4 和 16×8 分别是 46.7%、22.6% 和 6.2%。这正是 Section 5.4 用 `-log_view` 在整个正式矩阵上得到的次序，MAP 在事件核算层面与之一致。花在 OpenMP runtime 里的时间则朝另一个方向走，为 48.3%、69.3% 和 76.5%，而且那不是有效工作：被采样到的栈帧是 `cpu_relax`、`do_spin` 和 `__futex_wait`，也就是无事可做、正在等待的线程。两个 build 在重叠处都一致：四个共同的格子里，MPI 比例相差最多 0.9 个百分点，OpenMP 比例最多 4.3 个百分点，且没有系统性的偏向。

以上全部受 MAP 自身的局限约束。每格只有一份 profile，因此没有观测区间。它的开销大且不均匀（单独测得计时阶段为 +35%，`VecScatterEnd` 上约为三倍），而且它对 OpenMP 的插桩会扰动正在被测量的 barrier 行为，因此这些绝对百分比高估了未插桩运行会呈现的数值。

在这些局限之内，这张表回答了 Chapter 4 无法回答的一个问题，但只是在会计层面上，而不是机制层面上。随每 rank 线程数增长、并且必须先赚回来混合执行才划算的那份成本，是空耗在 OpenMP runtime 里的：增长的是 `cpu_relax`、`do_spin` 和 `__futex_wait` 上的时间，不是计算 kernel 上的时间。NUMA 放置仅凭拓扑就可以排除：八线程的 rank 会跨越共享 L3 的边界，但仍然装得进一个 16 核的 NUMA region，所以本文没有任何一种布局让 rank 跨越两个 NUMA region，四者在这点上不可能有差别（Table 2.2）。

这种增长也不是放置效应，因为它在固定布局下同样存在——此时一个 rank 的线程在每个节点数下都占据同样的核心、相对共享 L3 边界的位置也不变：固定 16×8 、把节点数从 2 提到 8，等待比例在 LibSci build 上从 53.5% 升到 76.5%，在基线 build 上从 57.8% 升到 77.5%，而每个线程化区域里的工作量在变少。

这条曲线另外两个特征，即随线程数单调上升、以及最大值恰好出现在 16×8 表现最差的地方，与同一个解释是一致的，也就是每个线程化区域里的工作量更少了。但它们与 cache 方面的解释同样一致：跨越两个 CCX 的 rank 可能产生负载不均，而这些不均会被同一批 barrier 汇集起来。profile 分不开这两者，因此线程究竟在等什么，留给 Section 7.3 的硬件计数器工作，Section 5.4.4 已说明这一点。Section 5.4 给出这笔交易的另一半，用的是完整矩阵，且不带 profiler。

最后一个观测是否定性的，但值得一提。线程化的 LibSci 链接进了生产 build，而 PETSc 的稀疏求解路径几乎不调用它：在任何一份 LibSci profile 中，BLAS 最多只占求解窗口 core-time 的 2.1%，而基线 profile 里根本没有采样到任何 BLAS 栈帧。因此线程化 BLAS 库无法解释 Chapter 4 测得的布局次序，这一点此前 Section 6.2 只能作为证据的局限来陈述，现在由 profile 数据确认。

5.4 瓶颈识别

Section 5.3 把硬件层面的问题留着没答，但未插桩正式运行的 `-log_view` event 表可以支撑一个更窄的问题，这个问题 Chapter 4 提出过却答不了：布局带来的效果，是否与求解中暴露于进程间同步的那部分比例相对应。这里解析了全部 264 次正式运行，因此证据覆盖的矩阵与扩展结果相同，且不带任何 profiler 开销。

5.4.1 测量什么

对每一次运行，取 RepeatedSolves 阶段中 halo 交换 event (`VecScatterBegin`、`VecScatterEnd`) 和 CG 全局归约 event (`VecTDot`、`VecNorm`) 的时间，除以该阶段的 `KSPSolve` 时间。有三条限制，仅靠 `-log_view` 一个也去不掉。第一，这些同步 event 是记录在 `MatMult` 及其同类内部的，因此这是一个暴露比例，而不是可加时间预算里的一项。把所有非容器 event 相加再对 `KSPSolve` 求比，会得到 116–372%，因为 `MatMult` 及其同类本身就是 halo 交换的容器：同一次等待在 `MatMult` 里算一次，在 `VecScatter` 那一对里算一次，在 `SFPack/SFUnpack` 里又算一次。

第二，每个 event 的时间都是各 rank 上的最大值，而不同 event 的最大值可能出现在不同的 rank 上，因此分子把一批最大值加了起来，这些最大值不一定都出现在提供分母的那个 rank 上。第三，`VecTDot` 和 `VecNorm` 本身也做局部浮点运算，所以它们的时间并非纯通信。

尽管如此，这个量在每一次运行里都是用同样的方式测出来的，因此在不同布局 and 不同节点数之间仍然可比。

5.4.2 Halo 交换主导暴露量，混合并行降低暴露量

Figure 5.2 和 Figure 5.3 给出结果。在两个维度上，对每一种布局、并且沿每一条弱扩展链单独来看，这个比例都随节点数单调上升；在 22 个规模-节点点中的 21 个上，它严格按 MPI rank 数排序：rank 越少，暴露越少。两条链分面画出，是因为它们并不重合：每核问题规模较小的那条链暴露更多，在相同节点数下二维最多高出 15.5 个百分点、三维 8.5 个百分点，这个差距与图本身想展示的布局间差距相当。图上画的比例是两类 event 之和，而这两类位于互不相交的调用路径上：halo 交换是从 `MatMult` 及其同类进入的，归约则直接来自 CG。

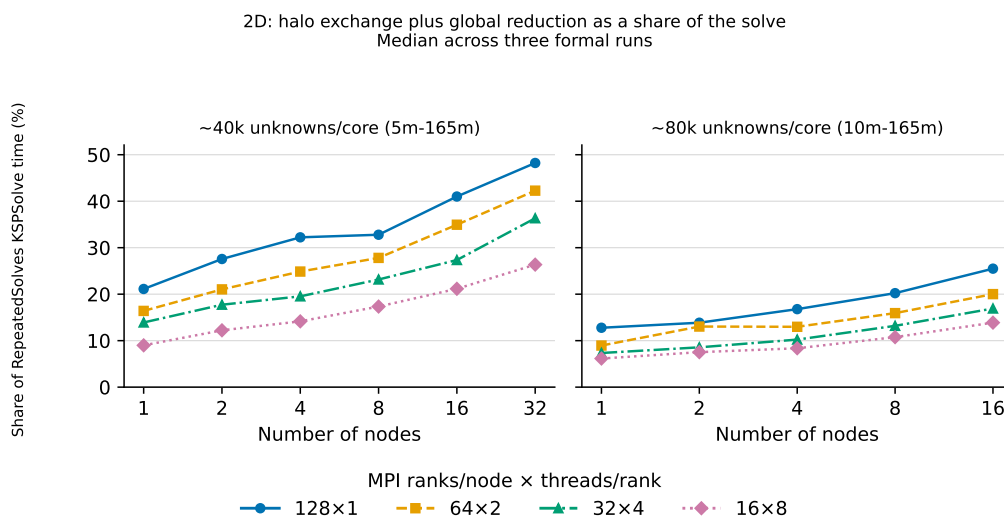


Figure 5.2: 2D share of `RepeatedSolves KSPSolve` time spent inside halo exchange and global reduction events, panelled by unknowns/core chain as in Chapter 4. Only the 40k chain reaches 32 nodes. Each point is the median of three repetitions. The ordering matches the time-per-iteration ordering in Figure 4.1 at every node count.

图上只画了这个总和。Table 5.3 在每个维度的 $\sim 40k$ 每核未知量链上，取四个有代表性的点把它拆开，这条链是能一直到 32 节点的那条。halo 交换在全部 16 个格子里都是较大的一项，二维为 1.7 到 2.9 倍，三维为 2.9 到 5.7 倍。在 32 节点 165m 上，纯 MPI 在二维为 halo 交换付出 34.3 个百分点、三维 70.9 个百分点，而付给归约的分别是 13.9 和 13.6。它们相加得到的总数，就是 Figure 5.2 和 Figure 5.3 最右端的 48.2% 和 84.6%。表中每一个点上三维的比例都远高于二维，这正是 Section 3.2 的分解所预测的：三维里 $\pm mn$ 这一项使得每行的两个 k 方向邻居在 L 降到 mn 以下之后都落到本 rank 之外，而二维里与之对应的 $\pm m$ 项最多只影响 $2m$ 行。

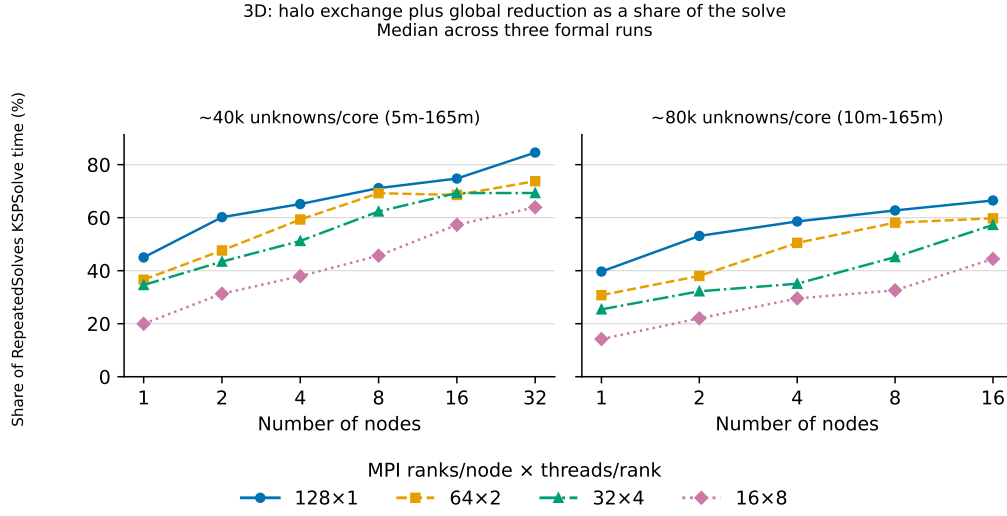


Figure 5.3: 3D share of `RepeatedSolves KSPSolve` time spent inside halo exchange and global reduction events, panelled and styled as in Figure 5.2. Note the vertical scale: flat MPI reaches 84.6% at the largest point.

Table 5.3: The exposure share of Figure 5.2 and Figure 5.3 split into its two event classes, at four representative points along the $\sim 40k$ unknowns/core chain of Table 4.2, for the 128×1 and 16×8 layouts. Halo is `VecScatterBegin` and `VecScatterEnd`; reduction is `VecTDot` and `VecNorm`. Their sum is the quantity plotted, so it is left to the figures. Each row is the repetition whose total is the median of its three, so both components come from the same run; the two are rounded independently and at one decimal need not sum to the plotted total (3D, 32 nodes, 128×1 : 70.9 and 13.6 against a plotted 84.6). Percentage points of `RepeatedSolves KSPSolve` time.

Case	Nodes (size)	128 $\times$ 1		16 $\times$ 8	
		Halo	Reduction	Halo	Reduction
2D	1 (5m)	15.7	5.4	6.1	2.9
2D	8 (41m)	23.4	9.4	11.2	6.1
2D	16 (82m)	28.6	12.4	13.6	7.6
2D	32 (165m)	34.3	13.9	16.4	9.9
3D	1 (5m)	35.1	9.9	15.3	4.6
3D	8 (41m)	53.0	18.1	38.8	6.8
3D	16 (82m)	59.0	15.8	46.5	10.8
3D	32 (165m)	70.9	13.6	52.4	11.5

5.4.3 降低暴露量是必要条件，但并不充分

光看次序解释不了结果，因为 16×8 在 Figure 5.2 和 Figure 5.3 的每一个点上暴露都最低，可它在二维 11 个点上的单次迭代时间却都是最差的。真正与性能对应的是这个差值的大小，而且要在单一布局内部来看，由此得到 Table 5.4 中的拟合。

记 S_ℓ 为布局 ℓ 图上那个 halo 加归约的比例， t_ℓ 为它的每次 KSP 迭代时间，两者取自同一个规模-节点点。Table 5.4 的前两列，是下面两个量在 11 个点上的取值范围，

$$\text{ExposureRemoved} = S_{128 \times 1} - S_\ell, \quad \text{TimeRatio} = \frac{t_\ell}{t_{128 \times 1}},$$

因此 $\text{ExposureRemoved} > 0$ 表示该布局比纯 MPI 暴露更少， $\text{TimeRatio} < 1$ 表示它更快。举例来说，按 Table 5.3，二维 16×8 在 32 节点 165m 上的 ExposureRemoved 是 $(34.3 + 13.9) - (16.4 + 9.9) = 21.9$ 个百分点。

打平点 (break-even) 是本文定义的一个量，由 a 和 b 算出，

$$\text{BreakEven} = \frac{1 - b}{a} = \frac{b - 1}{|a|}, \quad a < 0,$$

当 $a \geq 0$ 时不存在。这两个系数来自对每种布局那 11 对数据的最小二乘拟合，

$$\text{TimeRatio} = a \text{ExposureRemoved} + b.$$

其中 b 是截距，即该直线在 $\text{ExposureRemoved} = 0$ 处给出的时间比：如果一种布局的暴露比例和纯 MPI 一样，它相对纯 MPI 要付出多少，因此 $b - 1$ 就是纯粹线程化的代价。斜率 a 是 ExposureRemoved 每变化一个百分点时 TimeRatio 的变化量，因此 $|a|$ 就是一个百分点能把这份代价赚回多少。由于 $a < 0$ ，拟合直线正好在 ExposureRemoved 越过 BreakEven 的地方降到 1 以下。因此这一列是一个阈值：它表示一种布局要把暴露减少多少，线程化才开始划算，这个值越低就越早划算。拟合只是描述性的，因此单个点可能落在它错误的一侧。

r 一列是 Pearson 相关系数，由同样这 11 对数据算出，

$$r = \frac{\text{cov}(\text{ExposureRemoved}, \text{TimeRatio})}{\sigma_{\text{ExposureRemoved}} \sigma_{\text{TimeRatio}}},$$

也就是两者的协方差除以各自标准差之积。它与 a 同号，说明这些点贴合直线的紧密程度，因此它衡量的是打平点有多可靠，而不是另外增加一种效应。

Table 5.4: ExposureRemoved against TimeRatio, both relative to flat MPI at the same size-node point, the first in percentage points. The break-even value is the exposure reduction the fitted line requires to offset threading overhead; a and b are that line's slope and intercept, tabulated as $10^3 a$, so $(1 - b)/a$ reproduces the last column. *None* marks $a \geq 0$, where the line never descends to parity and no break-even exists. Eleven points per layout; the fits are descriptive only.

Layout	Exposure removed	Time ratio	r	$10^3 a$	b	Break-even
<i>2D</i>						
64×2	0.8–7.4	0.872–0.995	−0.67	−15.29	1.0178	1.2
32×4	5.3–13.7	0.937–1.096	−0.59	−11.13	1.1305	11.7
16×8	6.3–21.9	1.294–1.674	+0.00	+0.08	1.4313	none
<i>3D</i>						
64×2	1.9–15.1	0.768–1.022	−0.86	−21.60	1.0618	2.9
32×4	5.5–23.5	0.751–1.070	−0.90	−17.47	1.1270	7.3
16×8	17.4–31.1	0.899–1.122	−0.83	−15.82	1.4226	26.7

六个相关系数中有五个为负，三个绝对值超过 0.8：在某个点上一种布局减少的暴露越多，它在那里相对纯 MPI 就越快。 a 和 b 两列说明了为什么阈值会随每 rank 线程数急剧上升。竖着看 b ，也就是在还没减少任何暴露之前线程化本身的代价：它在线程数每次翻倍时都上升，而在 32×4 上它几乎与维度无关，1.1305 对 1.1270，尽管在最大的那个规模—节点点上纯 MPI 的暴露比例在二维和三维之间相差 36 个百分点。这一致性是合并拟合的性质，而且只对这一种布局成立，Section 5.4.4 说明把两条弱扩展链分开之后它还能保住多少。竖着看 $|a|$ ，也就是一个百分点的 ExposureRemoved 能换回多少：它每一步都在下降，而且三种布局在三维下都比二维大。打平点是两者之商，因此分子变大和分母变小叠加在一起，阈值上升得比单纯的线程化代价更快。这些阈值没有一个是外推出来的：凡是存在的，都落在该布局实际观测到的暴露减少量范围之内。

Figure 5.4 和 Figure 5.5 画出这些拟合，先放三维，因为它的三种布局表现一致，而整个论证所依赖的那个例外出现在二维。两张图都把每种布局的 11 个点按两条弱扩展链分开，实心对空心，并把表中列出的合并拟合用实线横贯其上。在三维，三种布局占据沿横轴依次错开的带状区域，每一种能达到的 ExposureRemoved 都比前一种更大，而在越过持平线之前需要的也更多；随着每 rank 线程数上升，这些带变得更平，这正是 a 那一列；而在每条带内部，实线和它的两条链走在一起。

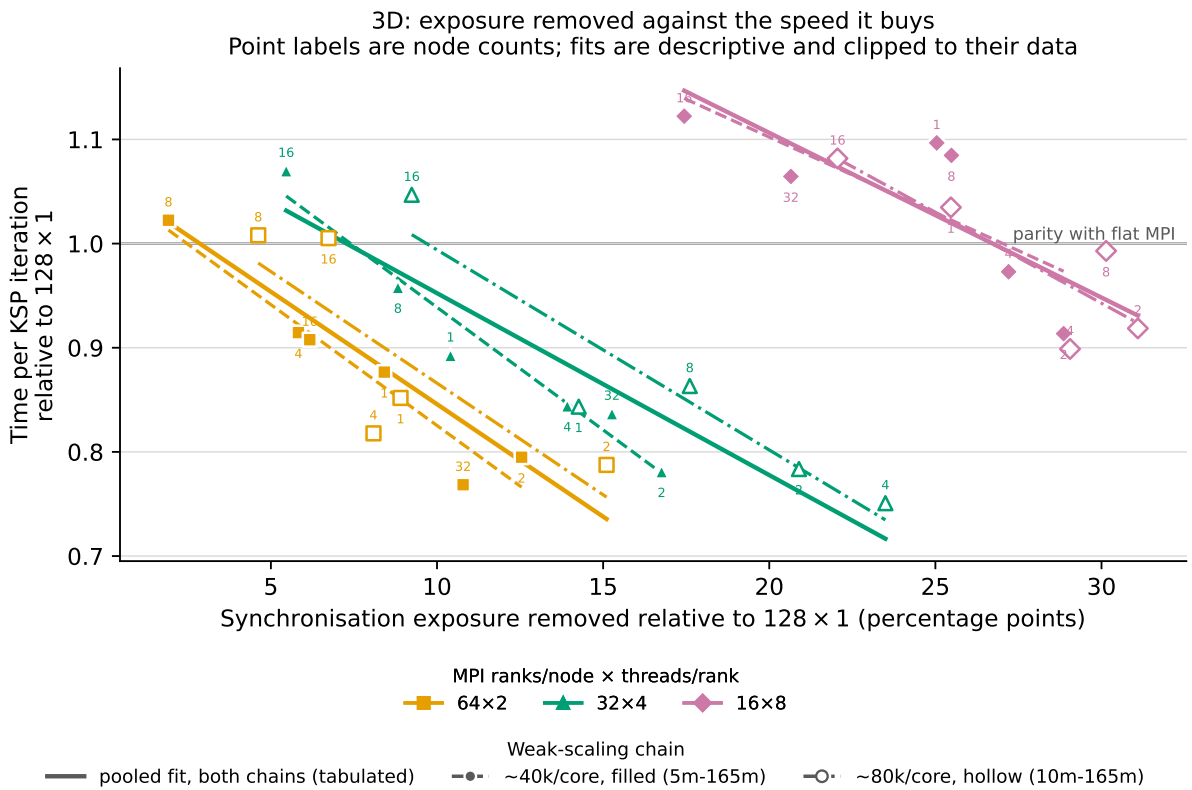


Figure 5.4: 3D exposure removed relative to flat MPI against the resulting time per iteration. Colour is the layout and point labels are node counts. The solid line is the pooled fit over all 11 points, the one Table 5.4 tabulates; the two broken lines split those points into their weak-scaling chains, dashed with filled markers for $\sim 40k$ unknowns/core and dash-dotted with hollow markers for $\sim 80k$. In 3D the three stay together, every layout’s per-chain break-evens falling within 2.4 points of its pooled value. Each line is clipped to the range it was fitted to, so none is extrapolated; the fits are descriptive, not predictive.

ExposureRemoved 与 TimeRatio 之间的这种关系，可以描述性地解释 Chapter 4 留下的两种现象。第一种：固定三维的某一布局和某一条链，沿节点数往上走，ExposureRemoved 和 TimeRatio 每一步都同时反向变化。在三维 $\sim 40k$ 链上， 32×4 从 1 节点到 32 节点的

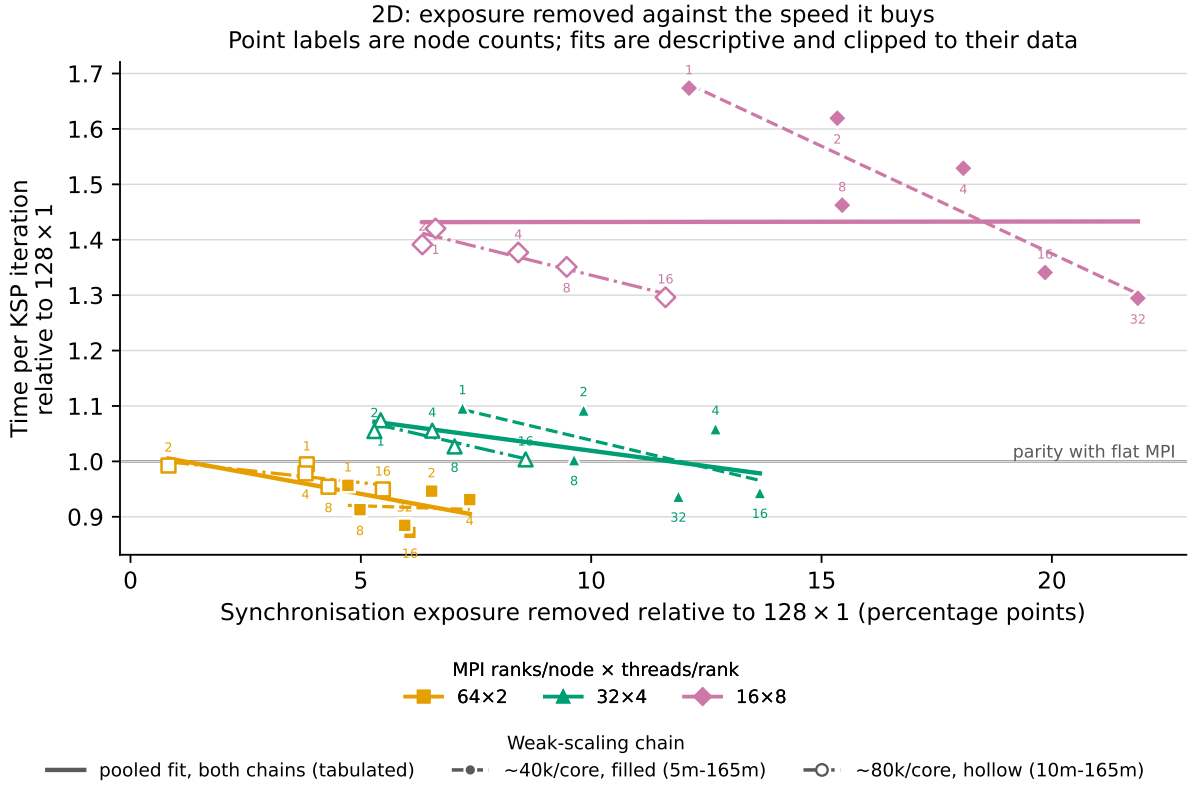


Figure 5.5: 2D exposure removed relative to flat MPI against the resulting time per iteration, styled as in Figure 5.4. Here the solid pooled fits are not summaries of their parts. For 16×8 the solid line lies flat while its two chains, correlating at -0.91 and -0.96 separately, descend steeply across it; 64×2 and 32×4 depart from their pooled fits as well. Note also that 16×8 lies above parity at every one of its 11 points, which is read off the markers and not off any fit.

ExposureRemoved 依次是 10.4、16.8、13.9、8.8、5.5 和 15.3 个百分点，TimeRatio 依次是 0.893、0.781、0.844、0.958、1.070 和 0.837：五次转移，五次反向，其中包括 32 节点上的那次回升——ExposureRemoved 上升 9.8 个百分点，把 7.0% 的劣势变成 16.3% 的优势。 64×2 和 32×4 在两条三维链上的全部 18 次转移都是这样。因此在这批测量之内，优势跟随的是并不单调的 ExposureRemoved。这说的是两者之差，而不是其中任何一个比例：沿这条链， 32×4 自身的暴露比例从 34.6% 单调升到 69.3%，纯 MPI 从 45.0% 升到 84.6%，正如 Section 5.4.2 所要求的，真正来回变化的只有两者之间的差。

第二种是阈值本身。在二维， 32×4 沿 $\sim 40k$ 链把自己 11.7 个百分点的打平点夹在中间：8 节点 41m 上 ExposureRemoved 为 9.6 个百分点，对应 TimeRatio 1.003；16 节点 82m 上为 13.7 个百分点，对应 0.944，正好在拟合所预言的位置越过持平线。而沿 $\sim 80k$ 链它的 ExposureRemoved 从未超过 8.6 个百分点，也从未越过持平线，最低只到 1.004。这个阈值在两个方向上都成立。

二维 16×8 这一行是反例，Figure 5.5 显示了它的构成：它合并拟合出的 a 是正的，所以实线是平的，表里也就给不出打平点；而穿过完全相同这 11 个点的两条虚线，每条链一条，都陡峭地向下穿过它。可见这种“平”是合并造成的假象。分开来看，两条链的相关系数在 6 个 $\sim 40k$ 点上是 -0.91 ，在 5 个 $\sim 80k$ 点上是 -0.96 ，属于整项研究中拟合最紧的几个。因此这种布局缺的不是 ExposureRemoved 与速度之间的关系，而是前者的量不够：每条链的直线要到超出该布局曾经达到过的最大 ExposureRemoved (21.9 个百分点) 之后才会到达持平线，所以两个阈值都无法在不外推的情况下写出来，也从未真正被达到。至于它在 11 个点上每一个都比纯 MPI 慢 29–67%，那是直接观测到的，不需要任何拟合。是别的什么成本占了主导。

Table 5.2 指出了那项成本：在相同的问题规模和节点数下， 16×8 花在 OpenMP runtime 里等待的 core-time 比任何其他布局都多，8 节点上是 76.5%，而 64×2 是 48.3%。因此这笔交

易的两半都被测到了，一半是在完整矩阵上不带 profiler 测的，另一半是在 11 个带 profiler 的格子上测的。没有被测到的是它们的和：被 profile 的量和未被 profile 的量来自不同运行上的不同工具，不能加成同一份预算。

5.4.4 证据能够与不能支持的结论

把一个维度全部 33 个混合测量点合在一起，会破坏 ExposureRemoved 与 TimeRatio 之间的关系，而且在两个维度上破坏的方式还不一样。二维合并后的相关系数是 +0.58，而其中两种存在打平点的布局分别是 -0.67 和 -0.59。三维则从三个绝对值都超过 0.8 的值降到 +0.10。

第一个可疑对象是 16×8 ，它的 ExposureRemoved 最大，同时也最慢。在三维这就够了：去掉它，合并后的数值回到 -0.75。在二维却不够，去掉它之后仍然是 +0.13。在那里， 32×4 在全部 11 个点上的 ExposureRemoved 都大于 64×2 ，而在全部 11 个点上都更慢。在两个维度上，布局之间的次序都与布局内部的次序相反， 16×8 只是其中最极端的一例。这种关系只在单一布局内部成立，因此一个合并的数值会误导人。

两条弱扩展链同样不能合并，这是第二个、且相互独立的效应：在三维它危害不大，每种布局按链算出的打平点都落在其合并值的 2.4 个百分点以内；而在二维，三种布局没有一个能原样保住， 16×8 更是连符号都反了过来。Table 5.4 仍然报告合并拟合，因为五个点和六个点太少，不足以单独拟合一条链，读它的各列时应当记住这一点。*b* 那一列之所以在两个维度间吻合，也是因为拟合是合并的：把链分开之后， 32×4 上的吻合会变松， 16×8 在 $\sim 40k$ 链上则完全不成立。两种合并都不改变哪些布局胜过纯 MPI，因为那是从测量里直接读出来的，不是从拟合里读出来的。

在 PETSc 的事件核算范围内，布局的优势与更低的 halo 交换暴露相对应，全局归约是次要的一项。MAP 则把与之相对的那份采样成本定位在 OpenMP 的等待路径上。这两个观测都不能排除在这些软件 event 底下还有内存或 cache 层面的机制。Section 6.1.4 列出候选，Section 7.3 给出用来判别的对照实验。

Chapter 6

讨论

6.1 回答研究问题

Section 1.2 的四个问题依次回答。RQ1 和 RQ2 依据 Chapter 4, RQ3 依据 Chapter 5, RQ4 依据前两者共同的局限。

6.1.1 RQ1: 线程后端是否缩短求解时间, 幅度多大?

是。混合执行在两个维度上都缩短了稳态求解时间。二维和三维的区别在于, 随着作业规模变大这份收益如何变化, 而不在于有没有收益。

在二维, 收益随节点数增长。沿两条每核未知量链, 每一种混合布局相对纯 MPI 的收益都随节点数增加, 六条序列中有四条每一步都在改善。 64×2 在全部 11 个规模-节点点上的单次迭代时间都短于纯 MPI, 优势从单节点的 4.3% 扩大到 16 节点的 12.8%。 32×4 在单节点慢 9.6%, 到 32 节点快 6.3%。因此, 在一两个节点上做出的布局比较, 无法预测 16 或 32 节点上会发生什么, 这就是二维结果的实用含义。

在三维, 收益更大, 却没有这样的趋势。混合布局的最大吞吐量增益是 33.2%, 二维为 14.7%, 且三分之二的混合点在单次迭代时间上快于纯 MPI。没有一条序列每一步都在改善: 优势在两到四个节点最大, 到八至 16 个节点消失, 在最大的那个点上又回来。按原始吞吐量计, 32×4 赢下 11 个点中的 6 个, 64×2 赢下 4 个, 纯 MPI 赢下 1 个。也就是说, 三维的最佳配比同时取决于问题规模和节点数, 而二维数据看不出这一点。

6.1.2 RQ2: 哪种配比更好, 结论如何随扩展场景变化?

每核工作量恒定时, 两个维度的有效范围都是每 rank 二到四个线程。二者之中哪个更好则取决于维度: 二维处处是 64×2 , 三维则视问题规模和节点数而定, 32×4 或 64×2 。

固定问题规模, 并不改变中等规模下哪些布局更好。它改变的是选错布局的代价, 从百分之几变成数倍, 因为各布局停止变快的节点数不同。二维没有一个停下来: 在 20.25M 未知量上, 四种布局从 1 个节点到 32 个节点之间的最大差距不超过 1.81 倍, 而且到最后仍在变快, 最后一次翻倍还有 1.11 到 $1.39 \times$ 的收益。三维则分道扬镳: 在配对的 20.35M 问题上, 纯 MPI 在 8 个节点后不再变快, 64×2 和 32×4 在 16 个节点后不再变快, 只有 16×8 到 32 个节点仍在改善。纯 MPI 在 32 个节点比它自己的最佳点还慢 $2.6 \times$, 因此在 4096 核上它与 16×8 的差距达到 $3.81 \times$ 。

每 rank 八个线程并不能在给定规模上跑得更快。它是在别的布局已经停下来的规模上还能继续变快。支持它的理由仅此一条。 16×8 在两组弱扩展实验中没有赢下任何一个原始吞吐量点, 而它在二维较高的弱扩展效率同样不能算作它的优点, 理由见 Section 6.2。它真正立得住的地方, 是 Section 4.1.2 的固定规模曲线。把问题压到约每核 4.9 千个未知量, 远低于 Section 3.4 准入区间允许的每核 2.5 万到 10 万, 16×8 就是 32 节点上最快的三维布局, 也是四种布局里唯一还在改善的。纯 MPI 到那时已经连续两次翻倍越跑越慢, 上面那个 $3.81 \times$ 就是这么来的。

一条布局建议, 如果不说明它是在哪种扩展场景下测出来的, 就没有意义。同样这 11 个规模-节点组合, 按每核工作量恒定来读, 给出的结论是有效范围为二到四个线程、 16×8 排在最

后；同样的求解器和同样的 build，放到固定规模的问题上，给出的却是 16×8 在 4096 核时是唯一还在改善的三维布局。

两种吞吐量视角都需要保留。原始吞吐量是这套求解器配置实际付出的时间，其中包含 GAMG 收敛次数的任何变化。单次迭代时间通过固定外层 KSP 迭代数把迭代次数的影响去掉，但即便如此，也不能让不同分解下的 GAMG 层次结构或通信量变得相同。

6.1.3 RQ3：求解过程的哪些部分与性能差异相对应？

Section 5.4 表明，混合布局在全部 22 个规模-节点点上暴露于 halo exchange 和全局归约的时间都少于纯 MPI，其中 halo exchange 是较大的一项。在同一种布局内部，它减少的暴露量与它快出多少是对应的 ($|r|$ 最高 0.90)。每 rank 线程数越多的布局，需要减少更多暴露量才能打平。这与二到四线程的有效范围以及三维不规则的排序一致，但并不能确定它们的成因。

这笔交易的另一半也测量了，用的是 20.25M 二维问题上的 11 份 Linaro MAP profile。在求解窗口内，core-time 中等待在 OpenMP runtime 里的比例随每 rank 线程数上升（8 个节点上分别为 48.3%、69.3% 和 76.5%），MPI 的比例则下降（46.7%、22.6% 和 6.2%）。这两个次序在两个不同的 PETSc build 上都成立。混合执行在 MPI 上省下的时间，有一部分又以线程互相等待的形式还了回去。这些采样与二维 16×8 表现不佳的结果一致，但不能排除 cache、带宽或负载不均的机制。Section 6.1.4 划出这条边界。

6.1.4 RQ4：这些趋势可推广到何种范围，还需哪些证据才能确定机制？

这些测量说明了发生了什么，但没有说明底层硬件里在发生什么，这一层仍然是敞开的。知道线程在等，并不等于知道它们在等什么。有三种成因同样能解释这些数据：PETSc 只对求解路径的一部分做了线程化；线程化区域可能太短，不足以把一次 barrier 的代价赚回来；内存带宽可能已经饱和。这三者在这里看上去是一样的。暴露量比例也有同样的问题：它无法把消息延迟、带宽、随 rank 数增长的开销，以及迭代中别处的负载不均分开。此外，被 profile 的量和未被 profile 的量来自不同的测量工具，不能相加成一份时间预算。本文没有确立任何硬件机制。

这些趋势能推广到多远，受 Section 6.2 所述范围的约束：两个结构化网格 Poisson 问题、一个 PETSc 版本和编译器环境、一套 CG+GAMG 配置、复用固定 operator 的重复求解，以及 PETSc 的默认行分解。在这个范围内，这些模式在两个维度、六种问题规模和两种扩展场景下都成立，与之相伴的暴露量次序和 OpenMP 等待次序在第二个 PETSc build 上也能复现。不过二维和三维并不是两次彼此独立的检验：两者用的是同一种行分解，而这种分解也正是它们表现不同的原因。Section 7.3 列出了可以把它们推得更远的做法：用硬件计数器查机制、用第二个和第三个固定问题规模检验饱和次序，以及把节点数推到 32 以上，去找到本实验只从下方框住的那些拐点。

6.2 有效性威胁

本实验矩阵覆盖两个结构化网格 Poisson 问题、一个 PETSc 版本和编译器环境、一套 CG+GAMG 配置，以及复用固定 operator 的重复求解。这里的任何结论都不必然迁移到非结构化矩阵、其他预条件器，或会调用稠密 BLAS 的求解阶段。其中最要紧的一条是复用：在 Section 3.8 那场对单次求解计时的实验里，setup 占了实测时间的中位数 69%，给出的布局排名也不一样。以一次性求解为主的应用，应当参照那个排名，而不是本文的排名。

最大的一条限制是数据分解。两个驱动程序都采用 PETSc 默认连续行块，因此子域是薄片而不是立方体，halo 遵循 Section 3.2 的 $\min(L, mn)$ 形式，而不是笛卡尔分解的 $N^{2/3}$ 。在 32 个节点、164.56 million 未知量时，这使纯 MPI 的 halo 在三维为 $2.03L$ 、二维为 $0.64L$ ，而同样大小的立方体子域约为 $0.18L$ 和 $0.02L$ 。布局之间的比较经得住这一点，因为四种布局跑的是同一份代码，而且模型在检验过的全部 48 个点上复现了它们的实测次序。经不住的是那个解释：混合执行减少的是这种分解所放大的那个 halo 的暴露量，而基于 DMDA 的驱动程序一开始的 halo 就要小得多。Section 7.3 给出了能判定还剩多少优势的那次实验。

三次重复给出的是观测到的波动范围，不是显著性检验。每核未知量的筛选规则使每个问题规模最多只有两个节点数，因此正式矩阵内的每一步强扩展都只跨越一次翻倍；把同样这些点

读成两条弱扩展链，只是把曲线拉长，并没有增加测量。固定规模扩展提供了真正的六点曲线，但每个维度只有一个问题规模，因此三维的饱和次序在其他规模上是否成立并未测过。它的波动也更大，最高达 11.1%，因为 32 节点下每次求解的时间只有几十毫秒。

弱扩展效率是相对每种布局自己的单节点时间算出来的，因此它衡量的是劣化程度而不是速度，而且会偏袒基线本来就差的布局。它必须和 Figure 4.1 与 Figure 4.3 一起读，绝不能单独使用。

按迭代次数相除，可以去掉二维 9–13 次、三维 6–7 次迭代的影响，但去不掉一次迭代内部随分解变化的 GAMG 工作量。单节点配置空间 sweep 的证据力更弱：它用的是默认的、依赖 rank 数的 block-Jacobi 求解器，而且其作业脚本里双横线的 `--ksp_rtol 1e-5` 从未被 PETSc 接受 (Section 3.3)，那些运行是在一个依赖网格的 tolerance 下收敛的，在 6400^2 时达到 2.4×10^{-10} 。因此它只确定搜索边界，仅此而已。本文不从中得出任何“默认求解器对 GAMG”的性能比较；确实依赖它的结论，都是在固定 rank 数下或按每次迭代成立的，此时被比较的各次运行共用同一个 tolerance。全部正式运行传这个选项时都用单横线。

正式运行链接了线程化 LibSci，但 Section 5.3 表明重复稀疏求解几乎不调用它，最多占求解窗口 core-time 的 2.1%，因此它是被排除掉的解释，而不是未获证实的解释。MAP 的计时仍然不进入扩展曲线，因为 profiler 开销大且不均匀；出于同样的原因，profile 中的比例只被当作布局之间的次序来读，而不是绝对成本。每个单元格只有一份 profile，且 128×1 一份也没有。关于硬件机制的主张仍在本文所确立的证据范围之外。

Chapter 7

结论与未来工作

7.1 结论

修正后的二维和三维实验包含 264 次正式运行，覆盖 1 到 32 个 ARCHER2 节点上的 88 种配置；配对的固定规模扩展又增加了 48 种配置上的 144 次运行。16 份试运行日志留档备查，但不计入统计。每一次正式运行都做一次预热求解和 19 次计时求解，并复用 GAMG 层次结构，每一次都对照 PETSc 自己报告的 build、布局、收敛情况和库做过验证。

每核工作量恒定时，每 rank 二到四个线程是两个维度上都可靠的范围，而每 rank 八个线程在两个维度上都没有竞争力。再往下，二维和三维的表现就不一样了。

在二维，布局带来的效果随节点数增长。每加一批节点，每一种混合布局相对纯 MPI 都在改善，六条“布局-扩展链”序列中有四条每一步都在改善。 64×2 在全部 11 个规模-节点点上的单次迭代时间都短于纯 MPI，单节点快 4.3%，16 节点快 12.8%；在 32 个节点、164.56M 未知量时达到 794.8 百万方程/秒，按原始吞吐量比纯 MPI 高 13.1%。 32×4 在单节点慢 9.6%，到 32 节点快 6.3%。

在三维，效果更大，却没有这样的趋势。最大的吞吐量增益是 33.2%，出现在四节点的 41m 上，三分之二的混合测量点在单次迭代时间上快于纯 MPI。但没有一条序列每一步都在改善，哪种布局最好也随问题规模和节点数变化。在 32 个节点的 165m 上， 64×2 达到 510.4 百万方程/秒，按原始吞吐量比纯 MPI 高 30.1%。而同样这个 165m 问题放在 16 个节点上时，每个核心要装的量是前者的两倍，那是整个三维实验中唯一一个纯 MPI 吞吐量四者最高的点。

固定问题规模时，同样这些布局拉开得远得多。二维曲线从 1 个节点到 32 个节点之间的差距始终不超过 1.81 倍，四种布局在 4096 核时仍在变快。三维曲线则分了家：四种里有三种不再变快、转而变慢，而发生这件事的节点数随线程数上升，纯 MPI 在 8 个节点， 64×2 和 32×4 在 16 个节点。只有 16×8 到 32 个节点仍在改善。纯 MPI 在那里比它自己的最佳点慢 2.6 $\times$ ，而在每一次弱扩展比较中都排最后的 16×8 ，在同一个点上比纯 MPI 快 3.81 $\times$ 。固定规模下真正把这些布局分开的，是各自在哪里停止变快，因此任何建议都必须写明它所依据的扩展场景。

性能剖析把这笔交易的两半都测到了：混合执行在通信上省下什么，又在线程化上付出什么。混合布局暴露于 halo exchange 和归约的时间更少；与此同时，11 份 MAP profile 显示，在 8 个节点上等待在 OpenMP runtime 里的 core time 比例随每 rank 线程数从 48.3% 升到 76.5%，两个 build 上都是如此。BLAS 最多只占求解期间 core time 的 2.1%。这些观测说明了时间在软件里花在哪儿，而不是硬件里什么造成了它。

这些测量取代了 launcher 时期无效的布局排名，Section 7.2 把它们转成具体建议。以上全部是在 PETSc 默认行分解上测得的，而 Section 6.2 表明这种分解会让 halo 变大，因此换成基于 DMDA 的驱动程序之后还能保住多少优势，尚未测过。本文的证据确立了这些性能模式，并指出了它们背后的软件成本，但没有指出底层的硬件机制。

7.2 ARCHER2 上 PETSc 的实践建议

二维默认用 64×2 。它在每一个实测点上的单次迭代时间都短于纯 MPI，单节点快 4.3%，16 节点快 12.8%，作业越大这个选择越稳妥。另有两处仍值得一试：单节点的小作业可以试纯 MPI，

它在 5m 和 10m 上的原始吞吐量分别高 5.9% 和 9.5%，因为少收敛一次 GAMG 迭代；大约八个节点以上可以试 32×4 ，它在这之下比纯 MPI 慢，在这之上比纯 MPI 快。

三维要在你实际打算用的节点数上同时测 32×4 和 64×2 ，不要从更小的节点数外推。 32×4 赢下的点更多，也给出了单点最大的优势，即四节点 41m 上的 33.2%；但在 32 节点 165m 上更好的是 64×2 ，它比纯 MPI 高 30.1%。而在 $\sim 80k$ 链的 16 节点上，没有任何一种混合布局能获胜。

不要默认用每 rank 八个线程。 16×8 在每核工作量恒定时的两个维度上都没有赢下任何一个原始吞吐量点，它在二维较高的弱扩展效率也不能算作证据 (Section 6.2)。只有一种情况值得一试：一个固定规模的三维问题，被推过其他布局停止变快的那一点。在 32 个节点、约每核 4.9 千未知量时，它是实测最快的布局，也是四种里唯一还在改善的。

这三条底下是同一件事：按作业实际所处的场景来选。每核未知量在 2.5 万以上时，各布局之间只差个位数百分比，二维用 64×2 、三维用 32×4 都是稳妥的默认。如果问题规模是固定的，而节点数是为了缩短求解时间被往上推的，那么这个选择值几倍的差距，而且能继续变快的是线程数最多的布局。在单节点上、或在每核工作量宽裕的情况下跑出来的基准测试，回答不了后一种情形。

7.3 未来工作

MAP profile 表明线程在等待。要弄清它们在等什么，需要硬件计数器。在固定的问题规模和节点数下逐布局采集内存带宽和 cache miss 计数，可以把 profile 分辨不出来的三种候选成因区分开：PETSc 只对求解路径的一部分做了线程化、线程化区域太短不足以抵偿一次 OpenMP barrier，以及带宽在所有核心忙起来之前就已饱和。直接测量 PETSc 的线程化区域而不是对它们采样，甚至不需要计数器就能解决其中第一种。

被 profile 的矩阵也很薄：一个问题规模、生产 build 上的两个节点数、三种混合布局，每格一次运行。它没有用来锚定趋势的纯 MPI 那一格，因为 MAP 启动不了这种布局。在第二个问题规模上、以及在三维固定规模曲线不再变快的那些节点数上重做一遍，就能检验 OpenMP 等待比例究竟是预测了一种布局在哪里停止变快，还是只是跟着它一起上升。

下一件要改的是数据分解本身。把两个驱动程序改建在 DMDA 上，使子域是立方体而不是薄板，再把同样四种布局跑一遍，可以把 Section 6.2 中的三维 halo 减小约一个数量级，也能说明本文测得的混合优势有多少来自线程化、有多少来自 PETSc 的默认行分解。设置中其他部分都不需要改动，因此两场实验可以逐点对比。

固定规模的结果引出两项扩展。第二和第三个固定问题规模，可以说明三维各布局停止变快的先后次序，是布局本身的性质，还是 20M 未知量这一规模的性质。把节点数推到 32 以上，可以说明 32×4 是否继续变慢，并定位 16×8 停止改善的那个点，本实验从未触及。在两种布局跑得很接近的地方增加重复次数，可以用置信区间取代最小-最大区间。更进一步的实验应当纳入预条件器 setup、变化的 operator、非结构化矩阵、其他编译器环境，以及 GPU 执行。

Bibliography

- [1] Satish Balay et al. *PETSc/TAO Users Manual, Revision 3.24*. Tech. rep. ANL-21/39 Rev. 3.24. Argonne National Laboratory, 2025. DOI: 10.2172/2998643.
- [2] Satish Balay et al. “Efficient Management of Parallelism in Object-Oriented Numerical Software Libraries”. In: *Modern Software Tools in Scientific Computing*. Ed. by Eirik Arge, Are Magnus Bruaset, and Hans Petter Langtangen. Boston: Birkhäuser, 1997, pp. 163–202. DOI: 10.1007/978-1-4612-1986-6_8.
- [3] PETSc Development Team. *Changes: 3.22*. 2024. URL: <https://petsc.org/release/changes/322/> (visited on 07/22/2026).
- [4] MPI Forum. *MPI: A Message-Passing Interface Standard, Version 4.1*. Nov. 2023. URL: <https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf> (visited on 07/22/2026).
- [5] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*. Nov. 2021. URL: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf> (visited on 07/22/2026).
- [6] Holly Judge. “Effective Use of MPI+OpenMP on a Cray EX Supercomputer”. In: *Proceedings of the Cray User Group 2022*. 2022. URL: https://cug.org/proceedings/cug2022_proceedings/includes/files/pap104s2-file1.pdf (visited on 07/22/2026).
- [7] Michael Lange et al. “Achieving Efficient Strong Scaling with PETSc Using Hybrid MPI/OpenMP Optimisation”. In: *Euro-Par 2012: Parallel Processing Workshops*. Ed. by Julian M. Kunkel, Thomas Ludwig, and Hans Werner Meuer. Berlin, Heidelberg: Springer, 2013, pp. 11–20. DOI: 10.1007/978-3-642-38750-0_8.
- [8] ARCHER2 Service. *ARCHER2 Hardware*. 2026. URL: <https://docs.archer2.ac.uk/user-guide/hardware/> (visited on 07/22/2026).
- [9] ARCHER2 Service. *Performance Tuning*. 2026. URL: <https://docs.archer2.ac.uk/user-guide/tuning/> (visited on 07/22/2026).
- [10] Magnus R. Hestenes and Eduard Stiefel. “Methods of Conjugate Gradients for Solving Linear Systems”. In: *Journal of Research of the National Bureau of Standards* 49.6 (1952), pp. 409–436. DOI: 10.6028/jres.049.044.
- [11] Klaus Stüben. “A Review of Algebraic Multigrid”. In: *Journal of Computational and Applied Mathematics* 128.1–2 (2001), pp. 281–309. DOI: 10.1016/S0377-0427(00)00516-1.
- [12] Linaro Limited. *Linaro MAP*. Linaro Forge 25.0.4 documentation. 2025. URL: https://docs.linaroforge.com/25.0.4/html/forge/forge/introduction_to_forge/map.html (visited on 07/22/2026).
- [13] PETSc Development Team. *KSP Tutorial ex2: Solves a Linear System in Parallel with KSP*. `src/ksp/ksp/tutorials/ex2.c`, PETSc 3.24 source distribution. 2025. URL: <https://petsc.org/release/src/ksp/ksp/tutorials/ex2.c.html> (visited on 08/09/2026).

Appendix A

可复现性：仓库与脚本

本论文所附的寄存器包含基准来源,ARCHER2 工作脚本, 每一个原始的日志, 以及分析管道将一个变成另一个。

A.1 目录结构

Table A.1 gives the directory layout of the repository.

Table A.1: Repository layout. Raw logs and retrieved job CSVs are immutable evidence; everything under a *derived* heading is regenerated by the scripts.

Path	Contents
src/	Benchmark drivers (Section 3.2)
scripts/ksp/{2d,3d}/lib/	Slurm job scripts and problem-size tables
scripts/petsc_configure_archer2.sh	PETSc configure recipes for both builds
analysis/data/<campaign>/logs/	Raw <code>-log_view</code> output, immutable
analysis/data/<campaign>/csv/source/	Retrieved job CSVs, byte-for-byte
analysis/maps/<campaign>/	Linaro MAP profiles, immutable
analysis/data/<campaign>/csv/derived/	Derived: normalised and aggregated CSVs
analysis/data/<campaign>/tables/	Derived: reference tables
analysis/plots/<campaign>/	Derived: figures, as PNG and PDF pairs
analysis/scripts/<campaign>/	Analysis code and its unit tests
docs/dissertation_latex/	This document and <code>sync_figures.sh</code>

这些运动是以产生它们的研究命名的。这篇论文的结果来源于 `nproc_2d_libsci` 和 `nproc_3d_libsci`, 两者均持有在 `strong_scaling_20m` 下验证的重复求解矩阵和固定尺寸 Section 4.1.2的扩展。同样的二维运动也延续了早期 `multi_node`, `single_node_hybrid` and `single_node_pure_mpi` 运行集, 在配置空间时收集到相同的构造和驱动程序固定; 其中只有 `multi_node` 在文本中使用, 用于 Table 3.3. `size_grid` holds the single-node 配置空间扫描 Section 3.5 和 `profiling_2d_map_baseline` Section 5.3的基线 MAP 剖面图。之前 `ksp_2d_scaling`, `ksp_3d_scaling`, `mpi_only` and `fixed128_multinode` campaigns 保留作为来源, 不用于任何布局索赔, 理由如下: Section 3.3 and Section 6.2.

A.2 证据索引

本论文中每个编号的图和表都是从原始工作产出中逐一得出的. 6 个脚本。Table A.2 给出每个脚本的链条: 生成的脚本它, 它从文件脚本写, 和原始证据所消耗的脚本。改为检查任意数字, 运

行其行中命名的脚本并进行比较；检查数字的取而代之，直接读取原始证据，而原始证据未经修改保留。

Table A.2: Evidence index. Script paths are relative to `analysis/scripts/`, derived and raw paths to `analysis/`. Log counts are the files present, which for the 3D fixed-size campaign exceeds the 72 accepted runs because superseded job attempts are retained. The last two rows are read directly from a named file and have no derived stage.

Figures and tables	Generated by	Derived output	Raw evidence
Table 3.1, Table 3.2, Figure 3.2	<code>size_grid/ analyze_single_node_ sweep.py</code>	<code>data/size_grid/tables/ iterations_vs_ layout.csv, fullnode_cost_per_ iteration.csv</code>	<code>data/size_grid/logs/ 213 logs</code>
Table 3.3	<code>nproc_2d_libsci/ analyze_multinode_ thread_limit.py</code>	<code>tables/multinode_ thread_limit.csv</code>	<code>logs/multi_node/ 45 logs, reconciled against csv/derived/fullnode_ runs.csv</code>
Table 4.1, Table 4.4, Figure 4.1, Figure 4.2, Figure 4.3, Figure 4.4, Figure B.1, Figure B.2	<code>nproc_{2d,3d}_libsci/ analyze_repeated_ fullnode_logs.py</code>	<code>csv/derived/repeated_ fullnode_formal_ summary.csv</code>	<code>logs/repeated_ fullnode/ 140 logs per dimension</code>
Table 4.5, Table 4.6, Figure 4.5– Figure 4.7, Table B.1, Table B.2	<code>strong_scaling_20m/ analyze_strong_ scaling_20m.py, then emit_runtime_ grid.py for the two grids</code>	<code>csv/derived/strong_ scaling_20m/*.csv, tables/strong_ scaling_20m/*.csv</code>	<code>logs/strong_ scaling_20m/ 72 (2D) and 108 (3D) logs</code>
Table 4.3	<code>cross_dimension/ summarize_best_ layouts.py</code>	<code>tables/weak_scaling_ best_layouts.csv</code>	the two formal summaries above
Table 4.7	<code>cross_dimension/ summarize_cross_ dimension.py</code>	<code>tables/cross_dimension_ summary.csv</code>	joins the three rows above
Table 5.2, Figure 5.1	<code>map_profiles/ analyze_map_ profiles.py</code>	<code>csv/derived/map_ profiles/map_call_tree_ shares.csv</code>	<code>maps/ 12 MAP profiles, 11 usable</code>
Table 5.4, Figure 5.2– Figure 5.5	<code>nproc_2d_libsci/ analyze_event_ breakdown.py (covers both dimensions)</code>	<code>tables/repeated_ fullnode_event_ matrix.csv, repeated_fullnode_ exposure_fits.csv</code>	<code>logs/repeated_ fullnode/ as above</code>
Table 2.2, Table 3.4	read directly	—	<code>scripts/ksp/ diagnostics/ petsc_diagnostics. 14598711.out</code>
Table 2.3, Table 5.1	read directly	—	one named <code>-log_view</code> log each, cited in place

Table 2.1 and Table 4.2 state definitions rather than measurements and have 没有证据链: 第一个是执行模型的分类, 第二个是重述 Section 3.4的录取规则。Figure 2.1 转载自 ARCHER2

硬件文档。

A.3 重新生成结果

从寄存器根创建分析环境后 `python3 -m venv analysis/.venv` and `analysis/.venv/bin/pip install -r requirements.txt`:

```
# unit tests (30)
analysis/.venv/bin/python -m pytest -c analysis/pytest.ini analysis/scripts

# single-node configuration-space sweep -> Table 3.1, Table 3.2, Figure 3.1
analysis/.venv/bin/python \
    analysis/scripts/size_grid/analyze_single_node_sweep.py

# multi-node threads-per-rank restriction -> Table 3.3
analysis/.venv/bin/python \
    analysis/scripts/nproc_2d_libsci/analyze_multinode_thread_limit.py

# 2D and 3D repeated-solve campaigns -> Chapter 4 weak-scaling figures
analysis/.venv/bin/python \
    analysis/scripts/nproc_2d_libsci/analyze_repeated_fullnode_logs.py
analysis/.venv/bin/python \
    analysis/scripts/nproc_3d_libsci/analyze_repeated_fullnode_logs.py

# matched fixed-20m strong scaling -> Chapter 4 strong-scaling figures
analysis/.venv/bin/python \
    analysis/scripts/strong_scaling_20m/analyze_strong_scaling_20m.py \
    --dimension 2d
analysis/.venv/bin/python \
    analysis/scripts/strong_scaling_20m/analyze_strong_scaling_20m.py \
    --dimension 3d

# full layout-by-node runtime grids -> Table B.1, Table B.2
analysis/.venv/bin/python \
    analysis/scripts/strong_scaling_20m/emit_runtime_grid.py

# duplicate-batch reproducibility check -> Section 4.1.2
analysis/.venv/bin/python \
    analysis/scripts/strong_scaling_20m/compare_superseded_batches.py

# event-level breakdown and exposure fits -> Section 5.4
analysis/.venv/bin/python \
    analysis/scripts/nproc_2d_libsci/analyze_event_breakdown.py

# trade-off figures, panelled by chain -> Figure 5.4, Figure 5.5
# reads only the derived table above, so it needs no raw logs
analysis/.venv/bin/python \
    analysis/scripts/nproc_2d_libsci/plot_exposure_tradeoff_by_chain.py

# Linaro MAP call-tree shares -> Section 5.3
analysis/.venv/bin/python \
```

```
analysis/scripts/map_profiles/analyze_map_profiles.py

# weak-scaling best layout per point      -> Table 4.3
analysis/.venv/bin/python \
    analysis/scripts/cross_dimension/summarize_best_layouts.py

# cross-dimensional comparison            -> Table 4.7
analysis/.venv/bin/python \
    analysis/scripts/cross_dimension/summarize_cross_dimension.py

# copy the generated PDFs into this document's figures/ directory
bash docs/dissertation_latex/sync_figures.sh
```

每个脚本打印已验证的日志和汇总配置, 并退出非零检查失败。将文档从目录中构建为:

```
cd docs/dissertation_latex
biber draft
latexmk -pdf -interaction=nonstopmode -halt-on-error draft.tex
```

Appendix B

补充图表

B.1 弱扩展吞吐量

Figure B.1 and Figure B.2 give raw repeated-solve throughput 为 Section 4.2的弱缩运动. 吞吐量几乎呈线性上升节点在两个维度上沿两个链子计数这就是它们携带的结果无法进行布局比较: 在对数轴上, 四面布局与三面布局相差少许数量级, 因此 Figure 4.1和 Figure 4.3正常化每个布局与平面的 MPI 相同大小- 节点对。

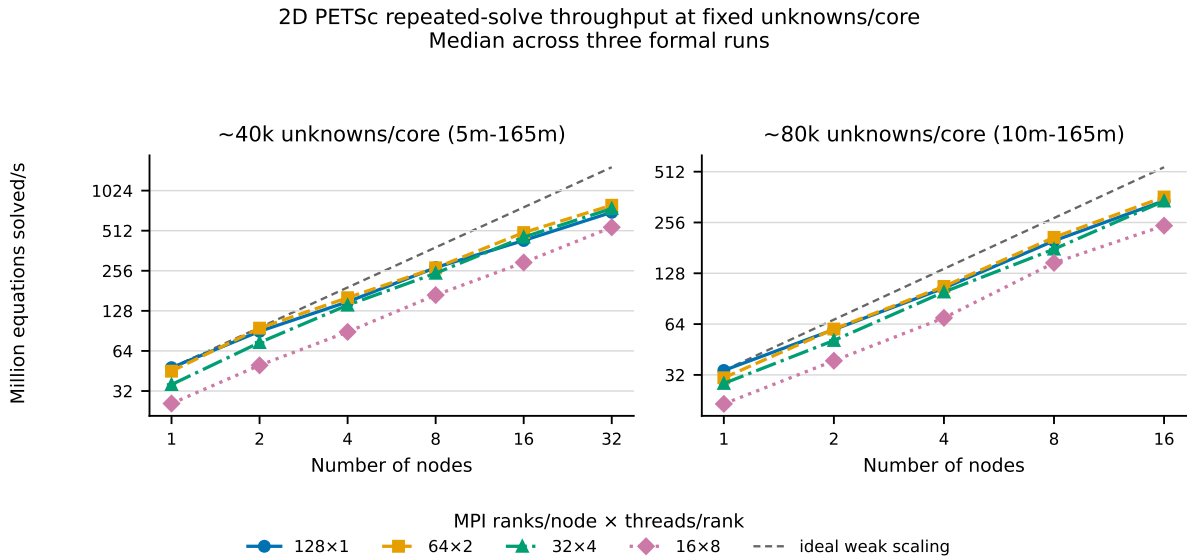


Figure B.1: Median 2D repeated-solve throughput along the two unknowns/core chains. Whiskers show the observed range of three formal runs. All layouts use 128 cores per node, so a fixed node count means a fixed core count.

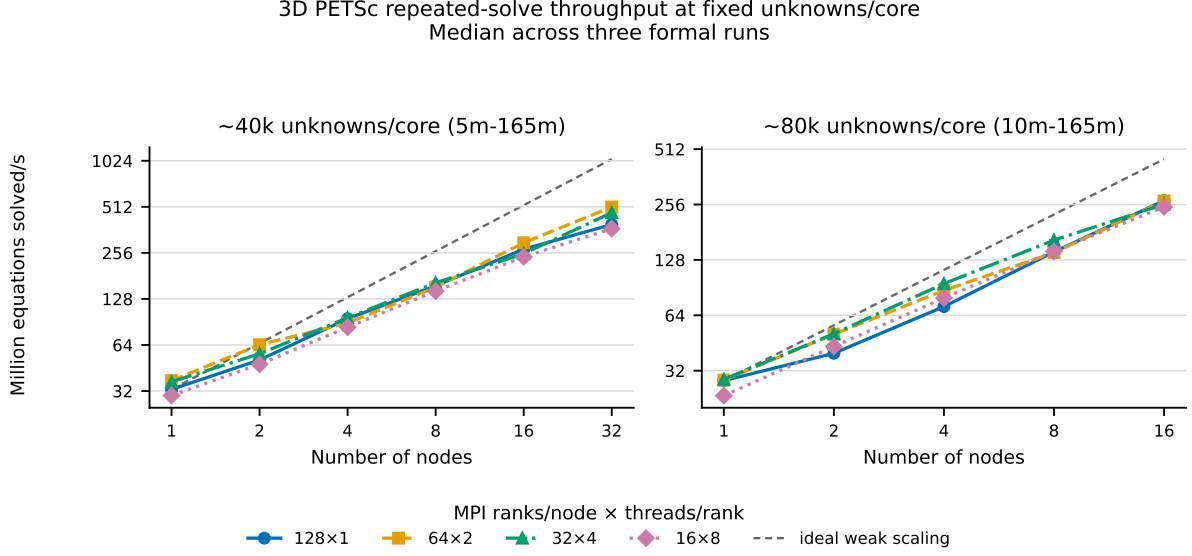


Figure B.2: Median 3D repeated-solve throughput along the two unknowns/core chains. Whiskers show the observed range of three formal runs.

B.2 固定规模运行时间网格

Section 4.1.2 presented these two campaigns as curves. Figure 4.5 显示整个范围内四个 2D 布局并落到 32 节点, 和 Figure 4.6 显示将三维布局分开, 作为其中的三个四个人到了最低点然后向上转

Table B.1 and Table B.2 give every point behind 这两个数字。4 章只列出最快的每个节点计数和 32 - 节点终点的布局, 所以这些网格是审查者需要检查运行时间读取任意一个数字。最后两列说明数字显示为形状: 每个布局增速超过最后双倍, 在曲线上的任何节点数上都显示最大三重奏的分布。

2D 网格在所有四行中单调落下。没有布局达到最小值在 32 节点内, 但最后双倍返回 1.11 到 $1.39\times$ 而不是 $2\times$ 早期的台阶, 所以曲线虽然没有转弯, 却在平坦。3D 网格在四排中三排向上, 在 8 个节点为 128×1 和 16 节点用于 64×2 和 32×4 ; 最后一个双倍分栏低于 1.00。正是那些布局越来越慢。

Table B.1: 2D fixed-size campaign, 4500^2 and 20.25M unknowns, median seconds per solve over three runs at every layout and node count. The final columns give the 16-to-32-node speedup and the largest observed three-repeat spread on that row. All layouts fill the node, so 32 nodes is 4096 cores in every row. Iteration counts range from 9 to 11.

Layout	Median s/solve at N nodes						16 $\rightarrow$ 32	Spread
	1	2	4	8	16	32		
128 $\times$ 1	0.7704	0.3399	0.1351	0.0558	0.0313	0.0282	1.11	7.9%
64 $\times$ 2	0.8260	0.3365	0.1265	0.0541	0.0348	0.0277	1.26	2.7%
32 $\times$ 4	0.8839	0.3944	0.1465	0.0632	0.0388	0.0317	1.22	10.8%
16 $\times$ 8	1.1325	0.5223	0.2272	0.0973	0.0568	0.0409	1.39	4.8%

Table B.2: 3D fixed-size campaign, 273^3 and 20.35M unknowns, median seconds per solve over three runs at every layout and node count. Columns as in Table B.1. A 16-to-32-node figure below 1.00 means the layout is slower at 32 nodes than at 16. Iteration counts are 6 or 7 throughout.

Layout	Median s/solve at N nodes						16→32	Spread
	1	2	4	8	16	32		
128×1	0.9066	0.5149	0.2208	0.0997	0.1776	0.2599	0.68	4.2%
64×2	0.8288	0.4041	0.2342	0.0936	0.0748	0.1703	0.44	3.2%
32×4	0.8022	0.4028	0.2145	0.1192	0.0652	0.0747	0.87	11.1%
16×8	0.9707	0.4718	0.2475	0.1330	0.1017	0.0681	1.49	7.6%

Appendix C

完整 log_view 摘录